

Report on

Intelligent Resume Screening and Skill Gap Analysis System

Submitted to:

Mr. Shah Dhruv Samir
(Assistant Professor)

University Institute of Computing

Submitted by:

Suraj Sharma (24MCI10198)
Adnan Irshad (24MCI10189)

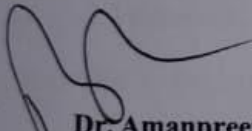


Chandigarh University, Mohali

BONAFIDE CERTIFICATE

Certified that this project report "**Intelligent Resume Screening and Skill Gap Analysis System**" is the Bonafide work of "**Suraj Sharma and Adnan Irshad**" who carried out the project work under my/our supervision.

SIGNATURE



Dr. Amanpreet Kaur
HEAD OF THE DEPARTMENT
(University Institute of Computing)



SIGNATURE

Mr. Shah Dhruv Smair
SUPERVISOR
(Assistant professor)
(University Institute of Computing)

Submitted for the project viva-voce examination held on 16 April, 2026



INTERNAL EXAMINER



EXTERNAL EXAMINER

Table Of Content

CHAPTER1: INTRODUCTION	1-10
1.1 Need Identification	
1.2 Problem Identification	
1.3 Task Identification	
1.4 Project Timeline	
1.5 Organization of the report	
CHAPTER 2: BACKGROUND STUDY.....	11-27
2.1. Evolution of Resume Screening Systems	
2.2 Proposed Solution	
2.3 Market & Literature Review	
2.4 Problem Definition	
2.5 Project Objectives	
2.6 Why Flask, Python & AI over Firebase	
2.7 Competitive Review	
CHAPTER 3: DESIGN AND PROCESS.....	28-49
3.1 Evaluation & Selection of Specifications	
3.2 Design Constraints	
3.3 User Experience Flow	
3.4 System Architecture	
3.5 API Design	
3.6 Implementation plan/methodology	
3.7 Interface Snapshots	
CHAPTER 4: RESULT ANALYSIS AND VALIDATION.....	50-53
4.1 Implementation of solution	
4.1.1 Backend Testing – Flask API routes	
4.1.2 Integration with Frontend	
4.1.3 Resume Matching Accuracy	

4.2 Testing & Validation

4.2.1 User Testing

4.2.2 Load Testing

4.2.3 Error Rate & Response Time

4.3 Result Interpretation

4.4 Continuous Feedback & Improvement

CHAPTER 5: CONCLUSION AND FUTURE WORK 54-59

5.1 Conclusion

Future Enhancements

APPENDIX A: 60-76

REFERENCES: 76-77

CERTIFICATION: 78

ABSTRACT

Recruitment these days is getting really tough because there are so many people looking for jobs online. Companies get flooded with applications, like hundreds for one spot, and recruiters have to sift through them all by hand. That takes forever and uses up a lot of time. Plus, it's subjective, you know, different people might pick different resumes, and there could be biases sneaking in. I think that is why we need something smarter to handle it, something automated that is fair and quick.

This project is about building an AI system to screen resumes. It uses data analysis to match resumes with job needs and rank people based on how well they fit. The idea is to cut down on all that manual work. It mixes AI with web stuff to make recruitment easier in today's world.

We made it as a web app. The back end runs on Flask, which handles the server stuff, routes, and logic. Front end is HTML, CSS, and Bootstrap for a nice-looking interface that works on phones too. Database is SQLite, simple for storing users, jobs, applications, all that. It keeps things straightforward, easy to set up, and we can add more later if needed.

There are parts for both sides, recruiters and applicants. Recruiters post jobs, list skills, qualifications, experience they want. Applicants sign up, log in, look at jobs, upload resumes. Once uploaded, especially PDFs, it pulls out the text using some libraries. Then cleans it up, fixes formatting, gets it ready for AI. That step turns messy data into something usable, I guess.

The main smarts come from TF-IDF and cosine similarity. TF-IDF turns words into numbers, weighing how important they are in the resume versus everything else. Then cosine checks how similar the resume is to the job description, gives a score. It also matches skills directly, compares what is in the resume to the job. Combines them with weights for a final score, ranks candidates. Better than just keywords, feels more complete.

One cool thing is anonymizing resumes. It strips out names, emails, phones, anything personal that might bias. So it just looks at skills and experience, makes it fairer. That tackles a big issue in hiring, promotes equal chances. Some might say it's not perfect, but it helps a lot.

Beyond screening, there is interview stuff built in. Recruiters schedule, send questions, grade answers in the app. Applicants do interviews right there. Keeps everything in one place, smooth workflow.

We tested it a bunch, functional tests, user ones, even load to see if it handles traffic. It works well, cuts time, ranks accurately. Processing is fast, errors are low because of good checks. Response times are okay, no big delays.

This system seems like a solid way to modernize hiring. It uses AI and web tech to fix old problems, makes decisions based on data, less bias. I might be oversimplifying, but it shows how AI can help real issues. Could lead to more improvements down the line.

LIST OF FIGURES

Fig 1	1
Fig 2	38
Fig 3	45
Fig 4	46
Fig 5	46
Fig 6	47
Fig 7	47
Fig 8	48
Fig 9	48
Fig 10	49
Fig 11	49
Fig 12	59



CHAPTER 1.

INTRODUCTION

1.1 Need Identification

In these days's rapidly evolving virtual panorama, recruitment has emerged as one of the maximum vital and aid-extensive procedures for companies throughout all sectors. With the rapid growth of on-line activity portals and multiplied accessibility to employment opportunities, the variety of job applicants has grown exponentially. As a end result, corporations regularly receive masses or maybe hundreds of resumes for a single process establishing. This overwhelming volume of applications makes the guide screening system no longer most effective time-ingesting but additionally inefficient and tough to manage. Recruiters are required to invest huge effort and time in reviewing each resume, which often ends in delays inside the hiring system and increased operational expenses. Guide resume screening is also noticeably at risk of human barriers and inconsistencies. Different recruiters might also compare the identical resume otherwise based totally on their person perspectives, revel in, or fatigue levels. This loss of standardization can bring about inconsistent selection-making and might result in the choice of less appropriate candidates even as overlooking extraordinarily certified people. Furthermore, human assessment is liable to subconscious bias, where factors including candidate name, gender, ethnicity, or academic background can also unintentionally impact hiring selections. Such biases can compromise the fairness and transparency of the recruitment procedure and negatively impact organizational variety and inclusivity. Similarly to these challenges, recruiters often battle to become aware of the most applicable candidates within a constrained time frame. The absence of green filtering and rating mechanisms makes it tough to healthy candidate profiles with task requirements as it should be. As corporations aim to remain competitive in fast-paced markets, delays in recruitment can result in ignored possibilities, lack of pinnacle expertise, and decreased productivity. Consequently, there may be a growing need for clever systems which could streamline and decorate the recruitment technique. The need for an automatic answer arises from more than one factors. First, there's a want to significantly lessen the manual effort concerned in resume screening, permitting recruiters to focus on greater strategic responsibilities inclusive of candidate engagement and decision-making. Second, enhancing the accuracy and consistency of candidate assessment is critical to make certain that hiring choices are based totally on goal criteria as opposed to subjective judgment. Third, disposing of bias from the recruitment manner is crucial for promoting fairness and equal opportunity among candidates.

Additionally, dashing up the recruitment cycle enables companies to speedy discover and onboard the quality candidates, thereby improving normal efficiency. Eventually, optimizing the recruitment method contributes to higher human resource control and enhances organizational performance. To address those challenges, the improvement of an AI-primarily based resume screening gadget becomes tremendously vital. Any such gadget utilizes advanced technologies, inclusive of herbal language processing (NLP) and gadget mastering strategies, to automate the analysis of resumes. Through converting unstructured resume information into dependent records, the machine can extract relevant details together with skills, experience, and qualifications and then compares this statistics with activity necessities to decide the suitability of each candidate. Furthermore, AI-driven structures are capable of processing big volumes of facts speedy and as it should be, making them surprisingly green compared to conventional methods. Strategies inclusive of TF-IDF and cosine similarity allow the machine to degree the relevance of a resume to a given task description, at the same time as ability-matching algorithms in addition enhance the assessment manner, and those structures also incorporate capabilities which includes anonymization, which gets rid of non-public identifiers from resumes to make sure unbiased assessment. In conclusion, the growing complexity and scale of contemporary recruitment procedures necessitate the adoption of intelligent and automated answers, and an AI-primarily based resume screening device not handiest improves efficiency and accuracy however also promotes fairness and transparency in hiring, and by way of leveraging superior technologies, corporations can remodel their recruitment strategies and successfully meet the needs of the virtual technology.

1.2 Identification of Problem

The recruitment industry faces several critical challenges that significantly hinder the efficiency, accuracy, and overall effectiveness of the hiring process. In today's digital era, where online job platforms and professional networks have become the primary medium for job applications, organizations are experiencing an unprecedented surge in the number of applicants for each job posting. While this provides access to a larger talent pool, it also introduces the major challenge of handling and processing an overwhelming volume of resumes. Recruiters are often required to review hundreds or even thousands of applications for a single position, making manual screening

not only time-consuming but also mentally exhausting. This excessive workload can lead to delays in shortlisting candidates and prolong the overall hiring cycle. Moreover, due to time constraints, recruiters may unintentionally overlook highly qualified candidates, thereby affecting the quality of hiring decisions.

Another significant challenge in the recruitment process is the lack of standardization in resume formats. Candidates present their information in a wide variety of styles, layouts, and structures, depending on personal preferences, industry norms, or available templates. Some resumes are highly detailed with extensive descriptions, while others are brief and minimalistic. Important information such as skills, experience, and achievements may be presented differently across resumes, making it difficult for recruiters to extract relevant data efficiently. This inconsistency complicates the process of comparing candidates objectively, as there is no uniform structure to evaluate them against a common standard. Consequently, manual screening becomes subjective, error-prone, and less reliable.

Bias in recruitment is another critical issue that continues to impact hiring practices across organizations. Even when recruiters aim to be fair, unconscious biases can influence their decisions. Factors such as a candidate's name, gender, age, educational background, or even geographic location may inadvertently affect the evaluation process. These biases can lead to unfair advantages or disadvantages for certain candidates, ultimately preventing organizations from selecting the most suitable talent based solely on merit and skills. Addressing this issue is essential not only for ensuring fairness and equality but also for promoting diversity and inclusion within the workplace.

In addition to these challenges, traditional recruitment systems lack intelligent mechanisms for evaluating resumes in relation to job requirements. Most existing systems rely heavily on basic keyword-based filtering techniques, which are limited in their ability to understand the context and relevance of information. For instance, a candidate may possess the required skills but describe them using different terminology, leading to their resume being overlooked by keyword-based systems. Similarly, keyword matching does not account for the depth of experience or the quality of skills, resulting in inaccurate or incomplete evaluation. The absence of advanced analytical capabilities makes it difficult to determine how well a candidate's profile aligns with a specific job role, thereby reducing the effectiveness of the recruitment process.

Another important issue is the inefficiency in communication between recruiters and applicants.

In many traditional systems, candidates do not receive timely updates regarding their application status, which can lead to uncertainty and dissatisfaction. Recruiters, on the other hand, may struggle to manage communication with a large number of applicants, resulting in delays and lack of transparency. This gap in communication negatively impacts the overall user experience and can harm the organization's reputation among job seekers.

Furthermore, managing and organizing large volumes of application data presents a significant operational challenge. Recruiters need to maintain records of resumes, job postings, application statuses, and interview details, often across multiple platforms or systems. Without a centralized and structured system, data management becomes complex and inefficient. This can lead to duplication of data, loss of important information, and difficulty in retrieving records when needed. As the number of applicants increases, these challenges become more pronounced, further complicating the recruitment process.

The cumulative effect of these issues highlights the limitations of traditional recruitment methods and underscores the need for a more advanced and automated solution. The key problems identified in the current recruitment process can be summarized as follows:

- Bias in hiring decisions caused by exposure to personal and demographic information
- Lack of intelligent and context-aware matching between resumes and job requirements
- Inefficient communication between recruiters and applicants, resulting in poor user experience
- Difficulty in managing, storing, and organizing large volumes of application data

1.3 Identification of Tasks

This project is trying to fix the problems with the way of recruiting people. It is doing this by making a list of things to do to create a recruitment platform that works well and is easy to use. The main goal is to make a website where people who are hiring and people who are looking for jobs can talk to each other easily. This website will use technology to look at resumes check out candidates and help make decisions. The system is being made to be automatic, accurate, fair and easy to use so it can keep up with the changing needs of recruiting people. The recruitment platform is what this project is about and it wants to make the recruitment platform work better, for everyone. At the foundation of the system is the development of a secure and intuitive user interface that supports both applicants and recruiters. Applicants are provided with functionalities to register, log in, and manage their profiles through a simple and user-friendly interface.

When you log in you can look at all the jobs that're available. You can read about what each job does and then apply for the ones you like. To apply you just need to upload your resume. This is easy to do because you can upload your resume in a format like PDF.

On the side the people who are hiring have a special page that helps them manage all the jobs they are hiring for. They can make a job posting by saying what the job is, what it entails what skills you need and how much experience you should have. This helps the system match the people with the right jobs. The people who are hiring can also. Delete job postings, see who has applied and keep track of how the hiring process is going. They can do all of this in one place, which makes things easier and faster for them. This means they do not have to use a lot of tools to get their job done. The job of hiring people becomes more efficient, for the recruiters. A crucial task of the system is the processing and analysis of uploaded resumes. Once a resume is submitted, the system extracts textual content using PDF processing techniques. This step transforms unstructured resume data into a machine-readable format suitable for further processing. The extracted text undergoes preprocessing, including cleaning, normalization, and removal of irrelevant characters or noise. This improves the quality and consistency of the data, ensuring more accurate analysis in later stages.

The system does an important job and that is to keep sensitive information secret. It removes things like names and email addresses and phone numbers from the information it gets. The system uses ways to find these things and take them out. This means that when people're looking at who to hire they do not know anything about the person that could make them biased. They just look at the skills and qualifications and experience of the person. The system helps make sure everything is fair. Everyone has an equal chance to get the job. This is very important when companies are hiring people nowadays. The system makes the hiring process fair and transparent. That is what the anonymization of sensitive information is all, about. The anonymization of information is a big part of making sure the hiring process is fair. After preprocessing and anonymization, the system applies AI-based techniques to evaluate candidate suitability. The implementation of TF-IDF (Term Frequency–Inverse Document Frequency) enables the conversion of textual data into numerical vectors, capturing the importance of words in the context of the document. Cosine similarity is then used to measure the degree of similarity between the resume content and the job description. This allows the system to determine how closely a candidate's profile matches the job requirements.

A combined scoring mechanism is used to generate a final match score, ensuring a balanced and comprehensive evaluation.

Another important task is the ranking of candidates based on their match scores. The system automatically sorts applicants in descending order of suitability, enabling recruiters to quickly identify the most relevant candidates. This significantly reduces the time required for manual shortlisting and enhances decision-making efficiency. Recruiters can review candidate profiles, analyze scores, and make informed decisions based on objective data rather than subjective judgment.

The system does a lot more than screening and ranking. It helps with managing applications too. Recruiters can pick the candidates say no to some applications or move candidates to the next step of the hiring process. The system has a part, for managing interviews. Recruiters can set up interviews give questions to candidates and look at their answers all in one place.

Applicants get messages answer screening questions. Do interviews using a special interface just for them. This means the whole process of finding and hiring someone is easy to manage. It all happens in one system. The recruitment process is managed efficiently within this system. The system manages the recruitment lifecycle.

The system has a part that manages the database to keep all the important information. This includes details about the users, job openings, applications and interviews. All this information is kept in a database that's easy to understand and use. This way the information is safe and easy to get when it is needed. The database is designed to grow with the system so it can handle information as more people use it.

The system is also made to be easy to use and understand. It has an clear interface that works well. The menus are easy to navigate. The layouts are simple. The system also sends out notifications at the time to help people use it better. Both the people looking for jobs and the recruiters get updates away, about the status of their applications interview schedules and other important things. This helps people talk to each other better and makes everything more open and clear. In conclusion, the defined set of tasks collectively contributes to the development of a comprehensive and intelligent recruitment system. By integrating secure user management, efficient data processing, advanced AI techniques, and streamlined workflow management, the system effectively addresses the limitations of traditional recruitment methods. It provides a scalable, reliable, and unbiased solution that enhances efficiency, improves accuracy, and supports better decision-making in the hiring process.

1.4 Timeline

Week	Phase	Objectives & Deliverables
Week 1–2	Research Requirement Analysis	& Identification of problem statement, study of existing recruitment systems, literature review, and finalization of technology stack (Flask, Python, SQLite, HTML/CSS).
Week 2–3	UI/UX Design	Designing wireframes for applicant and recruiter dashboards, login/register pages, job portal interface, and interview modules; defining user-friendly layout and navigation flow.
Week 3–5	Backend Development	Development of Flask application, creation of database schema, implementation of authentication system, job posting module, and resume upload functionality.
Week 5–6	AI Development	Algorithm Implementation of resume parsing, text extraction using PDF processing, anonymization using regex, and development of matching algorithm using TF-IDF and cosine similarity.
Week 6–7	System Integration	Integration of frontend templates with backend Flask routes, connecting database operations, and enabling smooth data flow between modules.
Week 7–8	Testing & Validation	Performing unit testing, integration testing, and user testing; validating resume matching accuracy; fixing bugs and optimizing performance.
Week 8	Documentation Finalization	& Preparation of final project report, screenshots, result analysis, and system deployment for demonstration.

1.5 Organization of the Report

This report is systematically structured into five comprehensive chapters, each addressing a specific phase of the project to ensure clarity, coherence, and a logical flow of information.

Chapter 1: Introduction provides a detailed overview of the project and establishes the foundation for the entire report. It begins with the identification of the need for an automated recruitment system in today's digital era, followed by a clear explanation of the problems associated with traditional resume screening methods. This chapter also outlines the tasks required to solve these problems and presents a structured project timeline that illustrates the development phases. Additionally, it defines the objectives of the system and highlights the importance of adopting AI-based solutions in recruitment. Overall, this chapter sets the context and significance of the project.

Chapter 2: Background Study focuses on the theoretical and practical understanding of the domain. It discusses the evolution of resume screening systems, starting from manual methods to modern AI-driven approaches. This chapter includes a detailed literature review of existing research and technologies related to recruitment automation and Natural Language Processing. It also provides a market analysis of current recruitment platforms and identifies their limitations. Furthermore, the chapter defines the problem statement clearly and presents the proposed solution adopted in this project. It also justifies the selection of technologies such as Flask, Python, and AI techniques over other alternatives. This chapter helps in understanding the research background and the motivation behind the system development.

Chapter 3: Design and Process is the most technical and detailed chapter of the report. It explains how the system is designed and implemented to achieve the desired objectives. This chapter includes the evaluation and selection of system specifications, design constraints, and overall system architecture. It provides a detailed description of the user experience flow for both applicants and recruiters, illustrating how users interact with the system. The chapter also explains the API design using Flask routes and describes how different modules communicate with each other. Additionally, it presents the implementation methodology, including resume processing, anonymization, AI-based matching, and database integration. Interface snapshots are included to demonstrate the visual design and usability of the system. This chapter provides a complete understanding of the internal working of the system.

Chapter 4: Result Analysis and Validation presents the practical outcomes of the project and evaluates the system's performance. It includes the implementation details and testing procedures carried out to ensure the correctness and reliability of the system. This chapter discusses backend testing of Flask routes, integration with the frontend, and evaluation of resume matching accuracy.

It also includes various validation techniques such as user testing, load testing, and analysis of error rates and response time. The results are interpreted to determine how effectively the system meets its objectives. Continuous feedback and improvements made during the development process are also highlighted. This chapter demonstrates the effectiveness and robustness of the system in real-world scenarios.

Chapter 5: Conclusion and Future Work provide a comprehensive summary of the entire project. It highlights the key achievements and contributions of the AI Resume Screening System in improving the recruitment process. This chapter reflects on how the system successfully addresses the identified problems and meets the defined objectives. It also discusses the limitations of the current system and suggests possible enhancements for future development. These enhancements include the integration of advanced AI models, external APIs, mobile application development, and scalability improvements. This chapter concludes the report by emphasizing the potential impact and future scope of the system.

Overall, the structured organization of this report ensures a clear and detailed presentation of the project. It enables the reader to understand each stage of development, from initial concept to final implementation and evaluation, in a systematic and comprehensive manner.

CHAPTER 2.

BACKGROUND AND LITERATURE REVIEW

2.1 Evolution of Resume Screening Systems

The process of recruitment has undergone a remarkable transformation over the years, evolving from traditional manual methods to highly advanced digital and AI-driven solutions. In the earlier stages, recruitment was entirely dependent on physical resumes submitted in printed form. Candidates were required to send hard copies of their resumes to organizations, and human resource professionals had to manually review each application. This process was not only time-consuming but also inefficient, as it required significant effort to sort, analyze, and shortlist candidates. Moreover, the lack of proper organization and storage mechanisms made it difficult to manage large volumes of resumes. As a result, this traditional approach lacked scalability and often led to delays in hiring decisions, ultimately affecting organizational productivity.

With the advancement of technology and the widespread use of the internet, recruitment processes began to shift towards digital platforms. Organizations started adopting online recruitment systems, where candidates could submit their resumes electronically through company websites or job portals. This transition significantly reduced the dependency on physical paperwork and improved accessibility for both recruiters and applicants. However, despite these improvements, the core challenge of manual resume screening remained unresolved. Recruiters still had to read and evaluate each resume individually, which became increasingly difficult as the number of applicants continued to grow. This resulted in increased workload and inefficiencies, especially for organizations receiving a large number of applications.

The introduction of Applicant Tracking Systems (ATS) marked a major milestone in the automation of recruitment processes. ATS platforms enabled organizations to store, organize, and manage resumes in a structured manner. These systems provided basic filtering capabilities, allowing recruiters to shortlist candidates based on specific keywords or criteria. While ATS significantly improved data management and reduced manual effort to some extent, it had its own limitations. The reliance on simple keyword matching often failed to capture the true context and relevance of a candidate's skills and experience. Candidates who did not use the exact keywords specified in the job description were sometimes overlooked, even if they possessed the required qualifications. This limitation reduced the effectiveness of ATS and highlighted the need for more intelligent solutions.

In recent years, the integration of Artificial Intelligence (AI) and Natural Language Processing (NLP) has revolutionized the field of recruitment, particularly in resume screening. AI-based systems are capable of analyzing large volumes of resumes quickly and accurately, extracting meaningful information, and understanding the context of the content. Techniques such as Term Frequency–Inverse Document Frequency (TF-IDF) and cosine similarity allow systems to measure the relevance of a resume with respect to a job description in a more sophisticated manner. These methods go beyond simple keyword matching by considering the importance and frequency of terms, thereby providing a more accurate evaluation of candidate suitability. As a result, AI-driven systems significantly reduce the time required for screening while improving the quality of candidate selection.

Furthermore, modern AI-based recruitment systems incorporate additional features such as skill extraction, ranking mechanisms, and predictive analysis. These capabilities enable recruiters to make data-driven decisions and identify the most suitable candidates more efficiently. Another important advancement is the inclusion of anonymization techniques, which remove personal identifiers from resumes to minimize bias and ensure fair evaluation. This promotes transparency and equal opportunity in the hiring process, which is essential in today's diverse and inclusive work environments.

The current project builds upon these technological advancements by developing an AI-powered resume screening system that integrates multiple functionalities into a single platform. It combines text analysis, skill matching, and automated ranking to provide accurate and efficient candidate evaluation. In addition to resume screening, the system also includes features such as anonymization to eliminate bias and an integrated interview management module to streamline the entire recruitment process. By incorporating these elements, the project aims to provide a comprehensive and practical solution that addresses the limitations of traditional and existing recruitment systems while meeting the demands of modern organizations.

2.2 Proposed Solution

The proposed system aims to develop an intelligent and automated platform that significantly enhances the efficiency, accuracy, and fairness of the recruitment process. In modern organizations, handling large volumes of job applications manually is both time-consuming and error-prone. To overcome these challenges, the system integrates multiple advanced components

such as resume parsing, anonymization, AI-based matching, and interview management into a single unified platform. This integrated approach ensures that all stages of recruitment are streamlined and managed efficiently within one system.

The core functionality of the system revolves around analyzing candidate resumes and comparing them with the requirements of a specific job role. When an applicant uploads a resume in PDF format, the system processes the file using PDF parsing techniques to extract textual content. This extracted text is then preprocessed by removing irrelevant characters, symbols, and formatting inconsistencies. The preprocessing step ensures that the data is clean, standardized, and suitable for further analysis, which improves the accuracy of the matching process.

A key feature of the proposed system is the implementation of anonymization techniques. Personal details such as names, email addresses, phone numbers, and gender-related terms are removed from the resume using regular expressions. This step plays a crucial role in ensuring that the evaluation process remains unbiased. By eliminating sensitive information, the system focuses entirely on the candidate's skills, qualifications, and experience, thereby promoting fairness and equal opportunity in the recruitment process.

The system employs a hybrid matching approach that combines both textual and skill-based analysis. It uses TF-IDF (Term Frequency–Inverse Document Frequency) vectorization to convert textual data into numerical form, allowing efficient comparison between resumes and job descriptions. Cosine similarity is then applied to measure the degree of similarity between the two texts. In addition to this, the system performs skill-based matching by identifying and comparing relevant skills present in the resume with those specified by the recruiter.

The final match score is calculated using a weighted formula that combines both textual similarity and skill matching. This balanced approach ensures that candidates are evaluated comprehensively rather than relying on a single factor. Based on these scores, the system implements an automated ranking mechanism. Candidates are sorted in descending order of their match scores, allowing recruiters to quickly identify the most suitable applicants. This ranking system significantly reduces the time and effort required for shortlisting candidates and supports faster decision-making.

Another important component of the system is the integrated interview management module. This module allows recruiters to create customized interview questions, schedule interview sessions, and evaluate candidate responses directly within the platform. Applicants can access a dedicated

interface to answer screening questions and participate in interviews. This integration eliminates Overall, the proposed system provides a complete and end-to-end recruitment workflow, covering all stages from job posting and resume submission to candidate evaluation and final hiring decisions. By combining automation, artificial intelligence, and user-friendly design, the system ensures improved efficiency, accuracy, and fairness. It addresses the limitations of traditional recruitment methods and offers a practical solution that meets the demands of modern organizations.

2.3 Market and Literature Review

The growing demand for efficient and intelligent recruitment solutions has led to the development of various systems and extensive research in the field of resume screening and talent acquisition. With the rapid increase in job applications and the need for faster hiring processes, organizations are continuously exploring automated tools to streamline their recruitment workflows. These tools aim to reduce manual effort, improve decision-making, and enhance the overall efficiency of hiring processes. However, despite the availability of such systems, many existing solutions still have limitations that restrict their effectiveness in real-world scenarios.

Several widely used platforms such as LinkedIn, Indeed, and other online job portals provide basic recruitment functionalities. These platforms allow recruiters to post job openings and filter candidates based on predefined criteria such as skills, experience, and keywords. While they are highly useful for connecting employers with job seekers, their filtering mechanisms are largely dependent on simple keyword matching and user-defined parameters. This approach lacks deep analysis and fails to capture the contextual meaning of resume content. As a result, candidates who may be suitable for a job but do not use specific keywords can be overlooked, while less relevant candidates may be shortlisted due to keyword presence alone.

Research in the domain of AI-based recruitment emphasizes the significant role of Natural Language Processing (NLP) techniques in improving resume screening. NLP enables systems to process and analyze human language in a meaningful way, allowing better interpretation of resume content. Among various techniques, TF-IDF (Term Frequency–Inverse Document Frequency) and cosine similarity have been widely recognized as effective methods for measuring textual similarity between documents. These techniques help in identifying important terms within a resume and comparing them with job descriptions to determine relevance.

In addition to NLP techniques, several academic studies have explored the use of machine learning algorithms for resume classification and ranking. Some approaches rely on supervised learning models that require labeled datasets to train the system, while others use unsupervised methods such as clustering and similarity-based grouping. Although these approaches can provide high accuracy, they often require large datasets, extensive training time, and significant computational resources. This makes them less practical for small-scale or real-time applications where simplicity and efficiency are important.

Another critical area of research in recruitment is bias reduction. Studies have shown that traditional hiring processes are often influenced by factors such as gender, ethnicity, and educational background, which can lead to unfair decision-making. Addressing this issue is essential for creating a fair and inclusive recruitment environment. AI-based systems that incorporate anonymization techniques have been proposed as a solution to this problem. By removing personal identifiers from resumes, these systems ensure that candidates are evaluated based solely on their skills and qualifications.

The proposed system builds upon these research findings by implementing a practical, efficient, and scalable solution. Instead of relying on complex deep learning models, the system utilizes TF-IDF and cosine similarity for effective resume-job matching. It also incorporates regex-based anonymization to eliminate bias and ensure fair evaluation. This approach strikes a balance between accuracy and computational efficiency, making the system suitable for real-world applications. By combining proven techniques with a user-friendly design, the proposed system addresses the limitations of existing solutions and provides a comprehensive framework for modern recruitment processes.

2.4 Problem Definition

Based on an in-depth background study and analysis of existing recruitment and applicant tracking systems, the problem can be formally defined as the requirement to design and develop an intelligent, automated, and scalable recruitment system that can efficiently analyze, evaluate, and rank candidate resumes according to specific job requirements. The system should aim to replicate and enhance human decision-making in recruitment by leveraging computational intelligence, while simultaneously improving speed, accuracy, fairness, and consistency in the hiring process. Traditional recruitment processes rely heavily on manual screening of resumes, which is not only

time-consuming but also prone to human bias and inconsistency. With the rapid increase in job applications due to digital job portals and online recruitment platforms, organizations face significant difficulty in handling and processing large volumes of candidate data effectively. Therefore, there is a strong need for an automated solution that can intelligently assist recruiters in filtering and selecting the most suitable candidates.

The complexity of this problem arises from several interconnected and technically challenging aspects that must be addressed in a unified system design.

One of the primary challenges is efficient handling of large-scale resume data. Modern recruitment environments often involve processing thousands of applications for a single job posting. The system must be capable of managing high data loads while ensuring minimal processing time and maintaining system responsiveness. This requires optimized data structures, efficient parsing techniques, and scalable architecture.

Another critical challenge is processing and interpreting unstructured resume data. Resumes are created in various formats such as PDF, DOCX, and plain text, and they often differ significantly in structure, layout, and terminology. Extracting meaningful information such as skills, work experience, educational background, certifications, and project details requires advanced preprocessing techniques and natural language processing (NLP). Without proper structuring, meaningful comparison between candidates and job requirements becomes extremely difficult.

A major aspect of the problem is accurate candidate-job matching and ranking. Simple keyword matching is insufficient because it ignores context, semantic meaning, and skill relevance. For example, different candidates may describe similar skills using different terminology. Therefore, the system must incorporate intelligent matching techniques such as semantic analysis, vector-based similarity models, or machine learning approaches to ensure that candidates are ranked based on actual relevance rather than superficial keyword overlap.

In addition, bias detection and elimination is a crucial requirement in modern recruitment systems. Human-driven hiring decisions can unintentionally introduce bias based on personal attributes such as name, gender, age, ethnicity, or educational institution. To ensure fairness and equal opportunity, the system must be designed to ignore or remove such sensitive attributes during evaluation and focus solely on professional qualifications, skills, and experience. This helps in promoting ethical AI practices and transparent decision-making.

Another significant challenge is ensuring transparency and explainability in candidate selection.

Recruiters should not only receive ranked results but also understand the reasoning behind the ranking. The system should provide clear explanations or scoring breakdowns that show how a candidate was evaluated. This increases trust in the system and allows recruiters to make informed decisions rather than relying on black-box outputs.

Furthermore, the system must provide a user-friendly and interactive interface for both recruiters and applicants. Recruiters require features such as job posting, candidate filtering, resume viewing, ranking dashboards, and analytics reports. Applicants, on the other hand, need a simple interface for resume upload, job search, application tracking, and profile management. A well-designed UI/UX ensures accessibility, reduces complexity, and enhances user satisfaction.

Scalability and performance optimization are also essential requirements. The system must be capable of adapting to increasing numbers of users, job postings, and resume datasets without performance degradation. This involves efficient backend design, database optimization, and potentially cloud-based deployment strategies.

Lastly, the system should be designed with data security and privacy considerations, ensuring that sensitive candidate information is securely stored and accessed only by authorized users. Compliance with data protection standards is essential in maintaining user trust and system reliability.

Summary of Key Challenges

- Efficient processing of large-scale resume datasets
- Extraction of structured data from unstructured resumes
- Semantic and intelligent matching of resumes with job descriptions
- Fairness and elimination of bias in evaluation
- Transparency and explainability of ranking results
- User-friendly interface for recruiters and applicants
- Scalability, performance, and system optimization
- Data privacy and security management

2.5 Project Objectives

The primary objective of this project is to design and develop an AI-based Resume Screening System that automates, streamlines, and enhances the overall recruitment process. In today's

highly competitive and fast-paced hiring environment, organizations are required to process a large number of applications within a limited time while maintaining accuracy, fairness, and consistency. Traditional recruitment methods are often inefficient and prone to bias, making it essential to adopt intelligent and automated solutions. This project aims to address these challenges by developing a system that integrates modern web technologies with advanced Artificial Intelligence (AI) techniques to create a reliable, scalable, and user-friendly recruitment platform. The system is designed with the objective of providing a seamless and efficient interaction between applicants and recruiters through a web-based interface. It enables applicants to easily register, log in, and apply for jobs by uploading their resumes, while recruiters can manage job postings, review applications, and make informed hiring decisions. By automating repetitive and time-consuming tasks, the system significantly reduces the workload of recruiters and accelerates the hiring process. One of the key objectives of the project is to implement effective resume parsing and text extraction techniques. The system must be capable of handling unstructured resume data and converting it into a structured format suitable for analysis. This involves extracting relevant information such as skills, qualifications, and experience from uploaded resumes using PDF processing methods. Accurate data extraction is essential for ensuring reliable analysis and improving the quality of candidate evaluation.

Another important objective is to develop an anonymization mechanism that ensures unbiased evaluation of candidates. The system removes sensitive personal information such as names, email addresses, and phone numbers before processing resumes. This feature helps eliminate unconscious bias in recruitment decisions and ensures that candidates are evaluated solely on the basis of their skills and qualifications. Promoting fairness and transparency in hiring is a critical goal of the system.

A central objective of the project is to implement AI-based matching techniques to evaluate the relevance of candidates for specific job roles. The system uses TF-IDF (Term Frequency–Inverse Document Frequency) to convert textual data into numerical representations and cosine similarity to measure the similarity between resumes and job descriptions. In addition to textual analysis, the system performs skill-based matching to identify the overlap between candidate skills and job requirements. A combined scoring mechanism is used to generate a match score, which provides a quantitative measure of candidate suitability.

The project also aims to develop a ranking mechanism that prioritizes candidates based on their match scores. This enables recruiters to quickly identify the most suitable candidates without manually reviewing every application.

1. Design a Web-Based Platform for Recruiters and Applicants

The system is intended to provide a comprehensive web interface that serves as a bridge between recruiters and applicants.

- **Recruiter Features:**

Recruiters can post job openings, view candidate applications, filter resumes based on relevance, and manage the recruitment workflow efficiently. The interface will include dashboards to track the status of applications and shortlist candidates.

- **Applicant Features:**

Applicants can register, log in, upload their resumes in PDF format, and track their application status. The interface is designed to be intuitive and user-friendly, minimizing complexity and ensuring accessibility for users with varying technical expertise.

- **Overall Goal:**

This objective ensures seamless communication and interaction, providing a centralized platform that reduces dependency on manual processes and multiple tools.

2. Implement Resume Parsing and Text Extraction Techniques

Resumes are often unstructured and inconsistent, making it difficult to extract relevant information for automated evaluation.

- **Approach:**

The system uses advanced text extraction techniques to parse resumes uploaded in PDF format. Key information such as skills, educational qualifications, professional experience, certifications, and contact details is extracted and converted into a structured format.

- **Benefit:**

Structured data allows the system to perform efficient analysis, comparison, and matching, which is not possible with raw unstructured resumes. This step is critical for enabling intelligent candidate evaluation.

3. Develop an Anonymization Mechanism for Unbiased Evaluation

To ensure fairness in recruitment, personal identifiers such as names, email addresses, phone numbers, gender, and other sensitive data must be removed.

- **Objective:**

The anonymization mechanism allows the system to evaluate candidates based solely on skills, qualifications, and experience, minimizing unconscious bias.

- **Impact:**

This promotes equal opportunity hiring and aligns with ethical AI practices and organizational diversity goals.

4. Apply TF-IDF and Cosine Similarity for Resume-Job Matching

Simple keyword matching is insufficient for accurate candidate-job alignment.

- **Technique:**

The system uses TF-IDF (Term Frequency-Inverse Document Frequency) to vectorize resume and job description text. Cosine similarity is then applied to measure the semantic relevance between the candidate's resume and the job requirements.

- **Benefit:**

This allows context-aware matching, capturing not just the presence of keywords but also their importance and relationship within the text, leading to more accurate shortlisting.

5. Perform Skill-Based Analysis and Calculate Match Scores

Beyond textual similarity, evaluating specific skills relevant to a job is crucial.

- **Method:**

The system identifies required skills from job descriptions and compares them with the candidate's listed skills. A match score is calculated using a weighted approach, combining textual similarity and skill alignment.

- **Outcome:**

This ensures a comprehensive evaluation that considers both experience/qualification descriptions and relevant skill sets, resulting in a more reliable assessment of candidate suitability.

6. Rank Candidates Based on Their Suitability

- **Mechanism:**

Once match scores are computed, the system automatically ranks candidates according to their relevance to the job.

- **Advantage:**

Recruiters can quickly identify the most qualified candidates, significantly reducing the time and effort needed for manual shortlisting while improving selection quality.

7. Provide Interview Management Features

The system extends beyond resume screening by supporting interview scheduling and management.

- **Features Include:**

- Creating and assigning interview questions
- Scheduling interviews
- Evaluating candidate responses
- Recording interview feedback

- **Impact:**

Integrating interview management ensures that the entire recruitment process—from application to hiring—is managed within a single system, providing better organization and reducing workflow fragmentation.

8. Ensure Secure Data Storage Using a Relational Database

Data security and integrity are essential in recruitment systems.

- **Implementation:**

A structured relational database stores user details, job postings, applications, and interview records securely.

- **Security Measures:**

Proper authentication, authorization, and access control mechanisms are implemented to maintain confidentiality and protect sensitive information.

9. Improve Overall Efficiency and Accuracy of Recruitment

The system aims to automate repetitive, time-consuming tasks while leveraging AI-based analysis to assist in decision-making.

- **Impact on Efficiency:**

Manual effort in screening, filtering, and shortlisting resumes is greatly reduced.

- **Impact on Accuracy:**

AI-assisted evaluation ensures consistent, objective, and highly accurate candidate assessments.

- **Overall Benefit:**

Organizations can shorten hiring cycles, improve candidate selection, and maintain high recruitment standards.

10. Deliver a User-Centric, Intelligent, and Scalable Solution

By combining automation, AI intelligence, and user-friendly design, the system provides a practical, end-to-end recruitment solution.

- **Scalability:**

Designed to handle large volumes of resumes and job postings without degradation in performance.

- **User-Centric Design:**

Ensures ease of use for both recruiters and applicants, improving adoption and satisfaction.

- **Comprehensive Goal:**

Ultimately, the system addresses modern recruitment challenges and enhances organizational hiring effectiveness.

2.6 Why Flask, Python & AI over Firebase

The selection of appropriate technologies plays a crucial role in determining the success, performance, scalability, and long-term maintainability of any software system. In the development of the proposed AI-based Resume Screening System, a careful and systematic evaluation of various technologies was conducted to ensure that the chosen stack not only satisfies the functional requirements of the system but also supports intelligent data processing, automation,

and future scalability. The system requires the ability to process unstructured textual data, perform real-time analysis, and integrate seamlessly with a web-based interface. After comparing multiple alternatives, a combination of Python, Flask, and AI-based Natural Language Processing (NLP) techniques was selected over other backend solutions such as Firebase due to its flexibility, efficiency, and strong support for machine learning-based applications.

Python has been chosen as the primary programming language for this project because of its simplicity, readability, and extensive ecosystem of libraries. It is widely regarded as one of the most suitable languages for data science, artificial intelligence, and machine learning applications. Python enables developers to write clean and concise code, which reduces development time and simplifies debugging and maintenance. It provides powerful libraries such as NumPy for numerical computations, pandas for data manipulation and analysis, and scikit-learn for implementing machine learning algorithms. In addition, Natural Language Processing libraries such as NLTK and spaCy allow efficient handling of text data, including tokenization, stop-word removal, and feature extraction. These capabilities are essential for processing resumes, which are typically unstructured and vary in format. Python's versatility also allows seamless integration of different modules, making it easier to combine data processing, AI algorithms, and web functionalities within a single application environment.

Another important advantage of Python is its strong community support and availability of extensive documentation. Developers can easily find solutions, libraries, and frameworks that accelerate development and improve system quality. Moreover, Python's compatibility with various platforms and its ability to integrate with other programming languages further enhance its suitability for building scalable and robust applications. Its role in this project extends beyond simple programming, as it forms the backbone of both backend logic and AI-driven processing. Flask has been selected as the web framework for developing the backend of the system due to its lightweight, flexible, and modular architecture. Unlike larger frameworks that include numerous built-in components, Flask follows a minimalistic approach, providing only the essential features required to build web applications. This allows developers to customize the system according to specific project requirements without unnecessary overhead. Flask supports routing, session management, API creation, and database connectivity, enabling smooth interaction between the frontend and backend components. Its simplicity makes it easier to understand, implement, and maintain, especially for projects that require rapid development and frequent updates.

One of the key strengths of Flask is its ability to integrate seamlessly with Python-based AI and machine learning models. Since the AI logic in this project is implemented using Python libraries, Flask serves as an ideal bridge between the user interface and the backend processing modules. It allows real-time processing of user requests, such as resume uploads and job matching, and ensures that results are delivered efficiently to the frontend. Additionally, Flask's modular structure supports scalability, enabling developers to extend the application with additional features such as advanced analytics, external API integrations, or cloud deployment in the future.

A major component of this project is the integration of Artificial Intelligence and Natural Language Processing techniques, which form the core of the resume screening system. The system utilizes TF-IDF (Term Frequency–Inverse Document Frequency) to convert textual data into numerical vectors by assigning weights to words based on their importance within a document and across a collection of documents. This technique helps in identifying the most relevant terms in a resume and job description. Cosine similarity is then used to measure the similarity between these vectors, providing a quantitative measure of how closely a candidate's profile matches the job requirements. This approach enables context-aware matching rather than simple keyword-based filtering, resulting in more accurate and meaningful candidate evaluation.

An important advantage of using TF-IDF and cosine similarity is their computational efficiency and simplicity. Unlike deep learning models, these techniques do not require large labeled datasets or extensive training processes. This makes them highly suitable for real-time applications where quick response times are essential. Additionally, they are easy to implement, interpret, and optimize, making them ideal for systems with limited computational resources. Despite their simplicity, these techniques provide reliable and effective results, making them a practical choice for this project.

In contrast, Firebase is a Backend-as-a-Service (BaaS) platform that offers features such as authentication, real-time database management, cloud storage, and hosting. While Firebase simplifies backend development and reduces the need for server management, it is not specifically designed for AI-based or NLP-driven applications. Implementing advanced resume screening functionalities using Firebase would require integration with external machine learning APIs or third-party services. This introduces additional complexity, increases dependency on external systems, and may lead to higher operational costs. Moreover, Firebase provides limited flexibility for implementing custom algorithms and complex data processing logic, which are essential

requirements for this project.

Another limitation of Firebase is its reduced control over backend operations. Since it is a managed service, developers have less flexibility in customizing the system according to specific needs. This can be a significant drawback when developing an application that requires fine-tuned control over data processing, algorithm implementation, and system behavior. In contrast, using Python and Flask allows complete control over the application logic, enabling the implementation of customized AI models and tailored recruitment workflows.

The advantages of selecting Python, Flask, and AI-based techniques over Firebase can be further understood in terms of flexibility, performance, and scalability. This technology stack provides full control over backend development, allowing the implementation of specialized algorithms designed specifically for resume screening. It reduces dependency on external services, thereby improving system reliability and minimizing long-term costs. The integration of AI capabilities within the application ensures that intelligent features can be developed and enhanced without relying on third-party platforms. Furthermore, the system can be easily scaled by upgrading the database, deploying on cloud platforms, or integrating advanced AI models without requiring major architectural changes.

In addition, this technology stack supports future enhancements such as incorporating deep learning models, integrating external APIs, and developing mobile or cloud-based versions of the system. The modular design of Flask combined with Python's flexibility ensures that new features can be added with minimal disruption to existing functionalities. This makes the system adaptable to changing requirements and technological advancements.

The selection of Python, Flask, and AI-based NLP techniques provides a strong, flexible, and efficient foundation for the development of the AI Resume Screening System. This combination enables the implementation of intelligent resume analysis, efficient data processing, and seamless user interaction. Compared to Firebase, it offers greater control, better support for custom AI algorithms, and enhanced scalability. As a result, the chosen technology stack is well-suited for building a comprehensive, reliable, and future-ready recruitment platform that meets the demands of modern organizations.

2.7 Competitive Review

The recruitment technology market has experienced rapid growth in recent years due to the increasing need for efficient, scalable, and intelligent hiring solutions. Organizations across various industries are adopting digital platforms to streamline their recruitment processes and manage the ever-growing volume of job applications. These platforms aim to reduce manual effort, improve candidate selection, and enhance overall hiring efficiency. However, despite these advancements, existing systems still face several limitations that restrict their effectiveness in delivering accurate and unbiased recruitment outcomes.

Traditional job portals represent the most basic form of digital recruitment systems. These platforms primarily serve as a medium to connect employers with job seekers by allowing companies to post job openings and candidates to submit their applications. While they provide accessibility and convenience, they lack advanced screening and evaluation capabilities. Recruiters using such platforms often rely on manual methods to review resumes, which becomes increasingly difficult as the number of applications grows. This not only increases the workload on recruiters but also leads to delays in the hiring process and potential oversight of qualified candidates.

To address some of these challenges, Applicant Tracking Systems (ATS) were introduced as a more structured approach to managing recruitment data. ATS platforms allow organizations to store, organize, and filter resumes using predefined criteria. They improve efficiency by enabling quick searches and automated sorting of applications. However, these systems primarily depend on keyword-based filtering techniques. This approach has significant drawbacks, as it does not consider the context or semantic meaning of the information present in resumes. Candidates who possess relevant skills but do not use specific keywords may be excluded, while others may be selected based solely on keyword matching without truly meeting the job requirements. As a result, ATS systems, although useful, do not fully solve the problem of accurate candidate evaluation.

In recent years, the emergence of AI-driven recruitment platforms has brought a new level of sophistication to the hiring process. These systems utilize machine learning and Natural Language Processing techniques to analyze resumes and identify suitable candidates more intelligently. They attempt to understand the context of textual data and provide better matching results compared to traditional systems. However, the adoption of such platforms comes with its own set of challenges. Many AI-based solutions require large datasets for training, involve complex configurations, and

often demand high computational resources. Additionally, they are typically offered as premium services, making them expensive and less accessible for small and medium-sized organizations. Another major limitation is that many of these systems do not adequately address the issue of bias in recruitment, as they may rely on historical data that contains inherent biases or fail to implement proper anonymization techniques.

The proposed system is designed to overcome these limitations by integrating multiple essential features into a single, cohesive platform. Unlike traditional systems that focus on limited functionalities, this system combines intelligent resume analysis, skill-based evaluation, and automated ranking mechanisms to provide a more comprehensive solution. The use of TF-IDF and cosine similarity enables accurate measurement of the relevance between resumes and job descriptions, ensuring that candidates are evaluated based on meaningful criteria rather than simple keyword presence.

In addition to textual analysis, the system incorporates skill-based matching, which further strengthens the evaluation process by comparing candidate skills with job requirements. This dual approach ensures that both contextual understanding and specific competencies are considered during screening. Another significant feature of the proposed system is resume anonymization, which removes personal identifiers such as names, email addresses, and gender-related information. This helps eliminate bias in the recruitment process and ensures that candidates are assessed purely on their qualifications and abilities, promoting fairness and inclusivity.

Furthermore, the system includes an integrated interview management module that allows recruiters to create interview questions, schedule interviews, and evaluate candidate responses within the same platform. This eliminates the need for multiple tools and provides a seamless workflow from application submission to final selection. The user interface is designed to be simple, intuitive, and accessible, ensuring ease of use for both recruiters and applicants.

The advantages of the proposed system can be understood in terms of its comprehensive functionality and efficiency. It provides a complete recruitment solution that includes intelligent matching, automated ranking, bias reduction, and interview management, all within a single platform. This integration reduces complexity, improves productivity, and enhances the overall recruitment experience. Additionally, the system is cost-effective and does not require extensive computational resources, making it suitable for real-world applications across organizations of different sizes.

CHAPTER 3.

Design and Process

3.1 Evaluation & Selection of Specifications

The development of the AI Resume Screening System required careful evaluation and selection of appropriate technologies, tools, and system specifications to ensure efficiency, scalability, and ease of implementation. The chosen specifications are based on the project requirements, performance considerations, and compatibility with AI-based processing.

Hardware Requirements

The system is designed to run on standard computing devices without requiring high-end or specialized hardware infrastructure. It is optimized to ensure smooth performance even on basic systems commonly available in academic and small organizational environments.

The minimum hardware requirements include:

- **Processor:** Intel i3 or higher (or equivalent) to efficiently handle web requests, data processing, and AI-based computations
 - **RAM:** 4 GB or more to support multitasking, resume parsing, and similarity calculations without performance lag
 - **Storage:** 500 MB free space for application files, libraries, temporary data, and database storage
- Internet connectivity for web access, enabling real-time interaction between users and the system
- These specifications ensure that the system remains lightweight, cost-effective, and easily deployable on a wide range of devices without requiring advanced computational resources. This makes the system suitable for academic projects, small-scale organizations, and prototype-level deployment environments.

Software Requirements

The software stack selected for this project has been carefully chosen to support both web development and AI-based resume processing functionalities. The system integrates multiple technologies to ensure efficiency, scalability, and ease of development. The software requirements include:

- **Programming Language:** Python, used for backend development, AI processing, and implementation of Natural Language Processing techniques
- **Framework:** Flask, a lightweight and modular web framework used for developing the backend structure, routing, and API integration
- **Database:** SQLite, a file-based relational database used for storing user data, job postings, resumes, and application records in a structured manner
- **Frontend Technologies:** HTML, CSS, and Bootstrap, used for designing a responsive, interactive, and user-friendly interface

Libraries Used:

- o **PyPDF2** (for extracting text from uploaded PDF resumes and converting unstructured data into readable format)
- o **scikit-learn** (for implementing TF-IDF vectorization and cosine similarity to perform intelligent resume-job matching)
- o **bcrypt** (for secure password hashing and protection of user credentials)
- o **FPDF** (for generating downloadable PDF reports such as shortlisted candidate lists and evaluation summaries)

Python has been selected due to its simplicity, high readability, and extensive ecosystem of libraries supporting Artificial Intelligence, Machine Learning, and Natural Language Processing. Flask provides flexibility and ease of integration between frontend and backend components, while SQLite ensures lightweight and efficient database management without requiring complex server setup. The combination of these technologies enables seamless integration of AI functionalities with web-based application features, ensuring both performance and maintainability.

System Requirements

The system is designed to support a complete, efficient, and intelligent recruitment workflow by integrating multiple functional modules within a unified platform. It aims to streamline the entire hiring process, from job posting and application submission to candidate evaluation and interview management. The design of the system focuses on meeting both functional and non-functional requirements, ensuring that it delivers high performance, reliability, scalability, and ease of use. By combining web technologies with Artificial Intelligence techniques, the system provides a comprehensive solution that enhances the efficiency and accuracy of modern recruitment

processes.

At its core, the system is built to support multiple types of users, primarily recruiters and applicants, each with distinct roles, permissions, and access levels. Role-based access control ensures that users can only perform actions relevant to their responsibilities, thereby improving security and maintaining system integrity. Applicants can create accounts, log in securely, and manage their profiles, while recruiters have access to administrative features such as job posting, application review, and candidate selection. This separation of roles ensures a well-organized workflow and prevents unauthorized access to sensitive functionalities.

The system provides applicants with the ability to upload resumes in PDF format and apply for job opportunities through a simple and intuitive interface. The resume upload process is designed to handle different file sizes and formats efficiently, ensuring a smooth user experience. Once uploaded, resumes are processed automatically without requiring manual intervention. This automation significantly reduces the effort required from both applicants and recruiters, making the application process faster and more convenient.

For recruiters, the system offers comprehensive tools to manage job postings and applications. Recruiters can create detailed job listings by specifying job roles, required skills, qualifications, and experience levels. These structured job descriptions serve as the basis for evaluating candidate suitability. The system allows recruiters to view applications, analyze candidate profiles, and manage the selection process through a centralized dashboard. This eliminates the need for multiple systems and improves overall workflow efficiency.

A key feature of the system is its ability to perform automated resume parsing. Once a resume is uploaded, the system extracts textual content and identifies important information such as skills, education, experience, and other relevant details. This process converts unstructured resume data into a structured format that can be easily analyzed. The extraction process is designed to handle variations in resume formats, ensuring consistent and accurate data processing. This capability forms the foundation for further analysis and intelligent decision-making.

The system incorporates advanced AI-based matching techniques to evaluate the relevance of candidates for specific job roles. It utilizes TF-IDF to convert textual data into numerical vectors, capturing the importance of words within the resume and job description. Cosine similarity is then applied to measure the similarity between these vectors, providing a quantitative assessment of how well a candidate's profile aligns with job requirements. This approach enables context-aware

analysis, going beyond simple keyword matching to deliver more accurate and meaningful results. Additionally, the system may incorporate skill-based comparison to further enhance evaluation accuracy.

Efficient data management is another critical aspect of the system. All information related to users, job postings, applications, and interviews is stored in a structured relational database. The database ensures data consistency, integrity, and easy retrieval, allowing the system to handle large volumes of information effectively. Proper indexing and query optimization techniques are used to improve performance and reduce response time. Security measures such as password hashing and controlled access are implemented to protect sensitive user data and maintain system reliability.

The system is designed to provide real-time interaction through a responsive web-based interface. Users can perform actions such as uploading resumes, applying for jobs, and viewing results without noticeable delays. The interface is built to be user-friendly and accessible across different devices, ensuring a smooth and consistent experience. Real-time feedback and notifications keep users informed about application status, interview schedules, and other important updates, thereby improving communication and transparency.

In addition to its functional capabilities, the system is designed with scalability and performance in mind. It can efficiently handle increasing numbers of users and large volumes of resume data without significant degradation in performance. The modular architecture allows for easy expansion and integration of additional features, such as advanced AI models, external APIs, or cloud-based deployment. This ensures that the system remains adaptable to future requirements and technological advancements.

Furthermore, the system improves recruitment efficiency by automating repetitive and time-consuming tasks such as resume screening, data extraction, and candidate ranking. This reduces the manual workload on recruiters and allows them to focus on more strategic aspects of hiring. By providing accurate and data-driven insights, the system enhances decision-making and ensures that the most suitable candidates are identified quickly and effectively.

3.2 Design Constraints

While designing and developing the AI-based Resume Screening System, several important constraints were carefully analyzed and considered to ensure that the final system remains practical, efficient, scalable, and suitable for real-world deployment. These constraints

significantly influenced the selection of algorithms, technologies, system architecture, and overall implementation strategy. Addressing these limitations at the design stage helped in building a balanced system that achieves a trade-off between performance, accuracy, security, and usability.

Technical Constraints

The system is built under certain technical limitations that arise due to hardware restrictions, data variability, and computational complexity. These constraints directly impact how data is processed, analyzed, and stored.

- **Limited computational resources for AI processing**

The system is designed to operate efficiently on standard computing devices without requiring high-performance GPUs or advanced processing units. Since deep learning models demand high computational power and large datasets, they were avoided in favor of lightweight techniques such as TF-IDF and cosine similarity. These methods provide efficient text representation and similarity computation while maintaining low memory usage and fast execution speed. This ensures that the system remains accessible to users with basic hardware configurations.

- **Dependency on accurate text extraction from PDF files**

A major technical challenge arises from the need to extract data from resumes uploaded in PDF format. PDFs often contain complex layouts, multiple columns, images, tables, and inconsistent formatting styles. These variations can lead to incomplete or inaccurate text extraction. To overcome this constraint, robust parsing libraries such as PyPDF2 are used along with preprocessing techniques like text cleaning, normalization, and noise removal to improve extraction accuracy.

- **Handling highly unstructured resume formats**

Resumes vary significantly in structure, terminology, and presentation style. Some candidates may provide detailed structured resumes, while others may submit minimal or poorly formatted documents. This inconsistency makes it difficult to extract uniform data fields. The system must therefore rely on flexible Natural Language Processing techniques to identify key entities such as skills, education, experience, and certifications, regardless of format differences.

- **Lack of standardized resume formats across users**

There is no universal format followed by all applicants, which creates inconsistency in data representation. This requires additional preprocessing effort to standardize extracted content

before analysis.

- **Real-time processing limitations**

Since the system aims to provide quick results, all AI-based computations must be optimized for near real-time performance. Complex models that require long processing times are avoided to ensure faster response generation.

Security Constraints

Security is one of the most critical aspects of the system, as it involves handling sensitive personal, academic, and professional information of users. Ensuring data confidentiality, integrity, and secure access is essential for maintaining user trust.

- **Protection of sensitive user data**

The system stores personal details, resumes, and application records, which must be protected from unauthorized access, leakage, or misuse. Proper data handling mechanisms are implemented to ensure confidentiality at all stages.

- **Prevention of unauthorized system access**

The system must ensure that only authenticated users can access specific functionalities. Role-based access control is implemented to differentiate between recruiter and applicant permissions. This prevents unauthorized modification or viewing of sensitive data.

- **Secure password storage using hashing algorithms**

Passwords are never stored in plain text. Instead, secure hashing techniques such as bcrypt are used to encrypt user credentials. This ensures that even if the database is compromised, passwords remain protected and unreadable.

- **Session management and authentication control**

User sessions must be securely managed to prevent session hijacking or unauthorized access. Proper session timeout mechanisms and secure login validation are required.

- **Risk of data breaches and cyber threats**

As a web-based system, it is exposed to potential security threats such as unauthorized access attempts, SQL injection, and data interception. Preventive measures are required to mitigate such risks.

Performance Constraints

The system must maintain high performance and responsiveness even under varying workloads and multiple user interactions. Performance optimization is essential to ensure smooth user experience.

- **Minimizing response time during resume processing**

The system must quickly process uploaded resumes and generate match scores without noticeable delay. Efficient algorithms and optimized code execution are essential to achieve fast response times.

- **Efficient handling of large datasets**

As the number of users, job postings, and resumes increases, the system must efficiently manage large-scale data. Poorly optimized queries or data structures may lead to performance degradation.

- **Concurrent user handling**

The system is expected to support multiple users simultaneously. Efficient request handling mechanisms are required to ensure that concurrent operations do not lead to system crashes or delays.

- **Database performance optimization**

As data volume increases, database queries must be optimized using indexing and structured schema design to ensure fast retrieval and storage operations.

- **Memory and processing optimization**

AI-based operations such as vectorization and similarity calculation must be optimized to reduce memory consumption and ensure smooth execution.

Usability Constraints

Usability plays a crucial role in determining how effectively users can interact with the system. A poorly designed interface can reduce system efficiency regardless of backend performance.

- **Designing a simple and intuitive user interface**

The system must provide a clean, structured, and easy-to-understand interface. Users with minimal technical knowledge should be able to navigate and operate the system without difficulty.

- **Ease of navigation for different user roles**

The system must clearly separate functionalities for recruiters and applicants. Recruiters should

easily access dashboards, job postings, and candidate evaluations, while applicants should be able to upload resumes and track applications effortlessly.

- **Consistency in user experience**

All pages and modules should maintain a consistent design structure, layout, and interaction style to avoid confusion and improve usability.

- **Clear feedback and notification system**

Users must receive proper feedback for every action performed in the system, such as successful login, resume upload confirmation, errors, or application status updates.

- **Accessibility considerations**

The system should be designed to be accessible across different devices and screen sizes, ensuring usability on desktops, laptops, and mobile devices.

Operational Constraints

In addition to technical and usability aspects, certain operational constraints also influence system behavior in real-world usage scenarios.

- **Dependency on internet connectivity**

Since the system is web-based, continuous internet access is required for full functionality. Poor connectivity may affect system responsiveness.

- **Server availability and uptime**

The system performance depends on server availability. Any downtime can affect user access and system reliability.

- **Data consistency and integrity management**

The system must ensure that all stored data remains consistent across different operations such as insertion, update, and deletion.

3.3 Design Flow

The system is designed to provide a seamless, efficient, and intuitive experience for both applicants and recruiters by clearly defining user interaction flows. These flows ensure that each user can perform their respective tasks in a structured and user-friendly manner, minimizing confusion and improving overall usability. The design focuses on simplifying complex recruitment processes while maintaining functionality and efficiency.

Applicant Flow

The applicant flow is designed to guide users through the process of job application in a clear and systematic way. Each step ensures that the applicant can easily interact with the system and complete the application process without difficulty.

- 1. User registers and logs in**

The process begins with the applicant creating an account by providing necessary details such as name, email, and password. Once registered, the user can log in securely to access the platform. This step ensures authentication and personalized access to system features.

- 2. Browses available job listings**

After logging in, the applicant can view a list of available job openings posted by recruiters. The system presents job details such as job title, description, and required skills, allowing users to explore opportunities that match their interests.

- 3. Selects a job and uploads resume**

The applicant selects a suitable job and uploads their resume in PDF format. The system accepts the file and prepares it for processing. This step marks the beginning of the automated evaluation process.

- 4. System analyzes resume and shows match score**

Once the resume is uploaded, the system extracts text, processes the content, and compares it with the job description using AI-based techniques. A match score is generated, indicating how well the candidate's profile aligns with the job requirements. This provides immediate feedback to the applicant.

- 5. User submits application**

After reviewing the match score, the applicant proceeds to formally submit the application. The system records the application and makes it available for recruiter review.

- 6. Receives notifications (selection/rejection/interview)**

The system keeps the applicant informed about the status of their application through notifications. These may include updates such as shortlisting, rejection, or invitation for an interview. This ensures transparency in the recruitment process.

- 7. Answers screening questions**

If shortlisted, the applicant may be required to answer screening questions provided by the

recruiter. This step helps in further evaluating the candidate's knowledge, skills, and suitability for the role.

8. Attends scheduled interview

Finally, the applicant participates in the interview as scheduled by the recruiter. This completes the application process and leads to the final hiring decision.

Recruiter Flow

The recruiter flow is designed to streamline the process of job posting, candidate evaluation, and hiring. It enables recruiters to efficiently manage applications and make informed decisions.

1. Recruiter registers and logs in

The recruiter creates an account and logs in to access the system. This ensures secure access and allows recruiters to manage their job postings and applications.

2. Posts job with required skills and description

The recruiter creates a job listing by providing details such as job title, description, and required skills. This information is used by the system to match candidate resumes accurately.

3. Views applications ranked by match score

Once applications are received, the system automatically analyzes and ranks candidates based on their match scores. This allows recruiters to quickly identify the most suitable candidates without manually reviewing every resume.

4. Reviews candidate details

The recruiter can view detailed information about each candidate, including extracted skills, experience, and match score. This helps in making informed decisions during the shortlisting process.

5. Selects or rejects candidates

Based on the evaluation, the recruiter can shortlist suitable candidates or reject those who do not meet the requirements. The system updates the status accordingly and notifies applicants.

6. Adds interview questions

For shortlisted candidates, the recruiter can create and assign screening or interview questions. This allows further assessment of the candidates before the final interview stage.

7. Schedules interviews

The recruiter schedules interviews for selected candidates by specifying date and time. The system ensures that applicants receive the necessary details and notifications.

8. Provides feedback and final hiring decision

After conducting interviews, the recruiter provides feedback and makes the final hiring decision. The system records the outcome and communicates it to the applicant, completing the recruitment process.

These well-defined user flows ensure that both applicants and recruiters can interact with the system efficiently. By simplifying each step and integrating automation, the system enhances user experience, reduces manual effort, and ensures a smooth and transparent recruitment process.

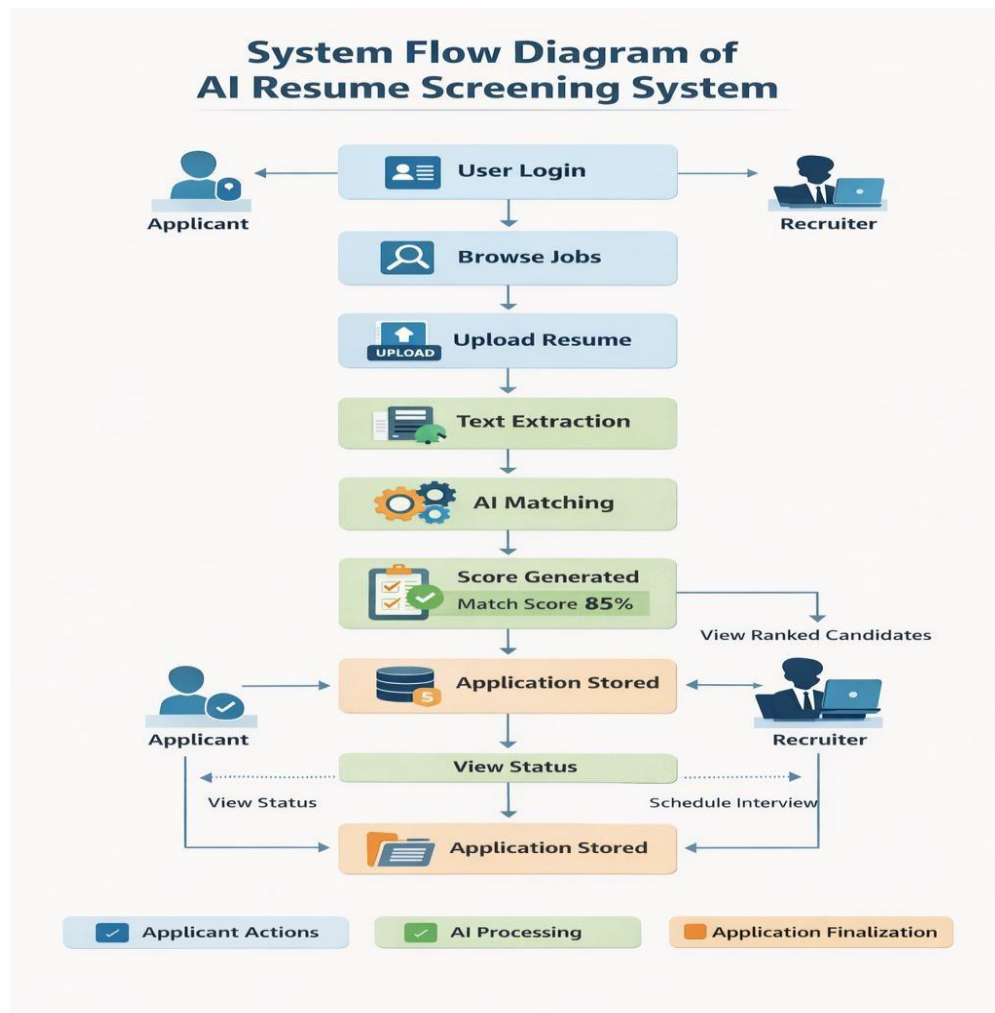


Fig.2 System Flow Diagram

3.4 System Architecture (Flask Server + Frontend UI + Database Schema)

The system is designed using a **three-tier architecture**, which ensures separation of concerns, better scalability, and ease of maintenance. This architectural approach divides the system into three main layers: Presentation Layer, Application Layer, and Data Layer. Each layer has a specific role and interacts with the others in a structured manner to ensure smooth functioning of the system.

1. Presentation Layer

The presentation layer is responsible for handling user interaction and displaying information to users. It acts as the front-end interface of the system where both applicants and recruiters interact with the platform.

- **HTML templates (UI screens)**

The user interface is developed using HTML templates that define the structure of different web pages such as login, registration, dashboard, job listings, and application forms. These templates ensure consistency and proper layout across the system.

- **CSS for styling**

CSS is used to enhance the visual appearance of the application. It helps in designing a clean, responsive, and user-friendly interface, ensuring that the system is accessible across different devices and screen sizes.

- **User interaction interface**

This layer manages all user inputs such as form submissions, button clicks, and navigation. It ensures smooth communication between the user and the backend system.

2. Application Layer

The application layer acts as the core processing unit of the system. It handles all backend operations, including request processing, business logic implementation, and AI-based analysis.

- **Flask backend**

Flask is used to build the backend of the system. It manages server-side operations and acts as a bridge between the frontend and the database.

- **Handles:**

- **Routing**

Flask routes manage different URLs and connect them to specific functions, enabling navigation between different pages and features of the application.

- o **Business logic**

This includes core functionalities such as user authentication, job posting, resume submission, application management, and interview scheduling.

- o **AI processing**

The application layer integrates AI techniques such as TF-IDF and cosine similarity to analyze resumes and match them with job descriptions. It also performs anonymization and scoring of candidates.

3. Data Layer

The data layer is responsible for storing and managing all the information used by the system. It ensures data persistence, security, and efficient retrieval.

- **SQLite database**

SQLite is used as the database system due to its simplicity and ease of integration. It stores all the necessary data in a structured format.

- **Stores:**

- o **Users**

Information related to applicants and recruiters, including login credentials and profiles.

- o **Jobs**

Details of job postings such as job title, description, and required skills.

- o **Applications**

Records of job applications submitted by candidates, along with their match scores and status.

- o **Interviews**

Information related to interview scheduling, questions, and candidate responses.

3.5 API Design (Skill-Recommendation, Roadmap Builder, Realtime Job Fetcher)

The system utilizes Flask routes as APIs to handle various functionalities and ensure smooth communication between the frontend and backend components. These APIs act as endpoints that receive user requests, process them through defined business logic, and return appropriate responses. By structuring the application using well-defined APIs, the system achieves modularity, scalability, and ease of maintenance. Each API is designed to perform a specific function within the recruitment workflow, ensuring organized and efficient system operation.

Authentication APIs

Authentication APIs are responsible for managing user registration, login, and access control. These APIs ensure that only authorized users can access the system and perform actions based on their roles.

- **/recruiter/login**

This API allows recruiters to log in by verifying their credentials. Upon successful authentication, recruiters gain access to features such as job posting, candidate evaluation, and interview management.

- **/applicant/login**

This endpoint enables applicants to securely log in and access functionalities such as browsing jobs, uploading resumes, and tracking application status.

- **/register**

This API handles new user registration by collecting user details, validating input, and securely storing the data in the database using encryption techniques.

These APIs play a crucial role in maintaining system security, enforcing role-based access control, and managing user sessions effectively.

Job Management APIs

Job management APIs enable recruiters to create, manage, and delete job postings while allowing applicants to view available job opportunities.

- **/recruiter/post-job**

This API allows recruiters to create and publish job listings by providing details such as job title, description, and required skills.

- **/recruiter/delete-job**

This endpoint enables recruiters to remove inactive or outdated job postings, ensuring that the system maintains updated information.

- **/applicant/jobs**

This API retrieves job listings from the database and displays them in a structured format for applicants to browse.

These APIs ensure efficient management of job-related data and smooth interaction between recruiters and applicants.

Application APIs

Application APIs handle the process of job applications and ensure that candidate data is properly recorded and processed.

- **/applicant/apply/<job_id>**

This API allows applicants to apply for jobs by uploading resumes and submitting required information. It also triggers resume analysis and AI-based matching.

- **/applicant/my-applications**

This endpoint allows applicants to view the status of their applications, including match scores, selection results, and interview updates.

These APIs simplify the application process and enhance transparency for applicants.

Interview APIs

Interview APIs manage the scheduling and execution of interviews within the system.

- **/recruiter/schedule-interview**

This API enables recruiters to schedule interviews by specifying date, time, and interview questions.

- **/interview-room/<room_id>**

This endpoint provides a virtual interface where candidates can participate in interviews and respond to questions.

These APIs integrate the interview process into the system, eliminating the need for external tools.

Notification APIs

Notification APIs ensure effective communication between the system and users.

- **/applicant/notifications**

This API retrieves notifications related to application status, interview schedules, and selection results, keeping applicants informed.

Overall, the API structure forms the backbone of the system by enabling efficient data flow, real-time interaction, and modular system design.

3.6 IMPLEMENTATION PLAN AND METHODOLOGY

The system is developed using a modular and incremental approach, ensuring that each component is built, tested, and validated independently before integration. This methodology reduces complexity, improves reliability, and allows early detection of errors. Each module performs a

specific function, and together they form a complete and efficient recruitment system.

Step 1: Database Setup

The development process begins with designing the database structure using SQLite. This database serves as the foundation for storing and managing system data.

- **users table**

Stores information about applicants and recruiters, including credentials, roles, and profile details.

- **jobs table**

Contains job postings with details such as job title, description, and required skills.

- **applications table**

Maintains records of job applications, including applicant ID, job ID, resume details, match score, and status.

- **interviews table**

Stores interview-related data such as schedules, questions, and candidate responses.

This structured design ensures efficient data storage, retrieval, and management.

Step 2: User Authentication

This step focuses on implementing secure access control mechanisms.

- Registration and login system

Users can create accounts and log in based on their roles.

- Password hashing using bcrypt

Passwords are encrypted to ensure security and prevent unauthorized access.

This module ensures data protection and controlled system access.

Step 3: Resume Processing

This module handles resume uploads and prepares data for analysis.

- File upload handling

Ensures resumes are uploaded in supported formats such as PDF.

- Text extraction

Extracts textual content using libraries like PyPDF2.

- Data preprocessing

Cleans and standardizes text for accurate analysis.

This step converts unstructured data into a structured format.

Step 4: AI Matching Algorithm

This is the core intelligence of the system.

TF-IDF Implementation

- Converts text into numerical vectors
- Assigns importance to words based on frequency

Cosine Similarity

- Measures similarity between resume and job description
- Generates a relevance score

This ensures accurate and efficient candidate evaluation.

Step 5: Skill Matching

Enhances evaluation by focusing on specific skills.

- Compare extracted skills with required skills
- Calculate match percentage

This ensures both contextual and skill-based analysis.

Step 6: Ranking System

Candidates are ranked based on their match scores.

- Sort candidates in descending order
- Identify top candidates quickly

This reduces manual effort and improves decision-making.

Step 7: Interview Module

Handles the final stage of recruitment.

- Add interview questions
- Schedule interviews
- Record candidate responses

This integrates the entire recruitment process within the system.

Step 8: Testing and Optimization

Ensures system reliability and performance.

- Debugging to fix errors
- Performance optimization for faster processing

- UI enhancement for better user experience

3.7 Interface Snapshots

This section includes screenshots of the system interface to demonstrate its functionality and design.

Key Screens

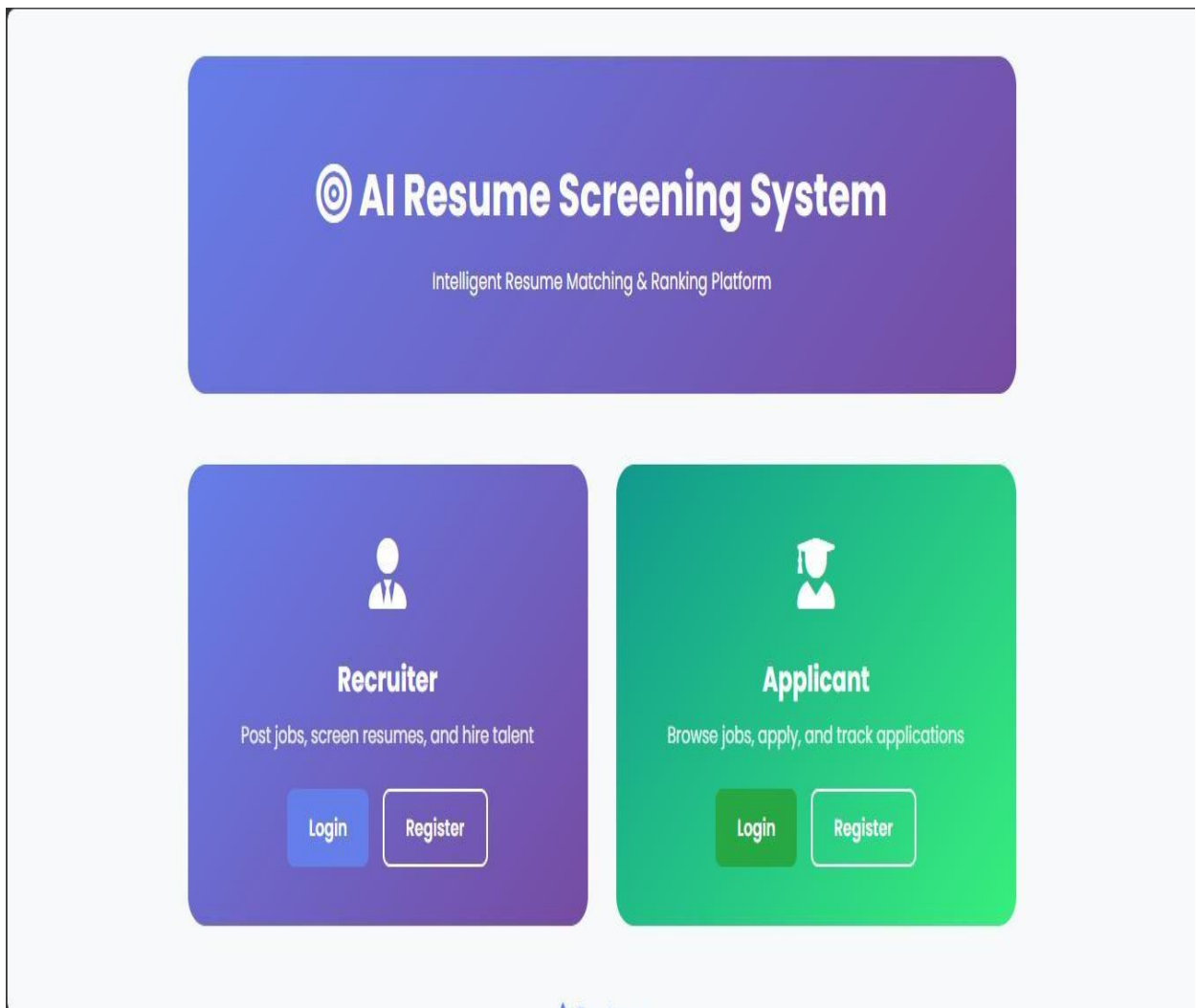
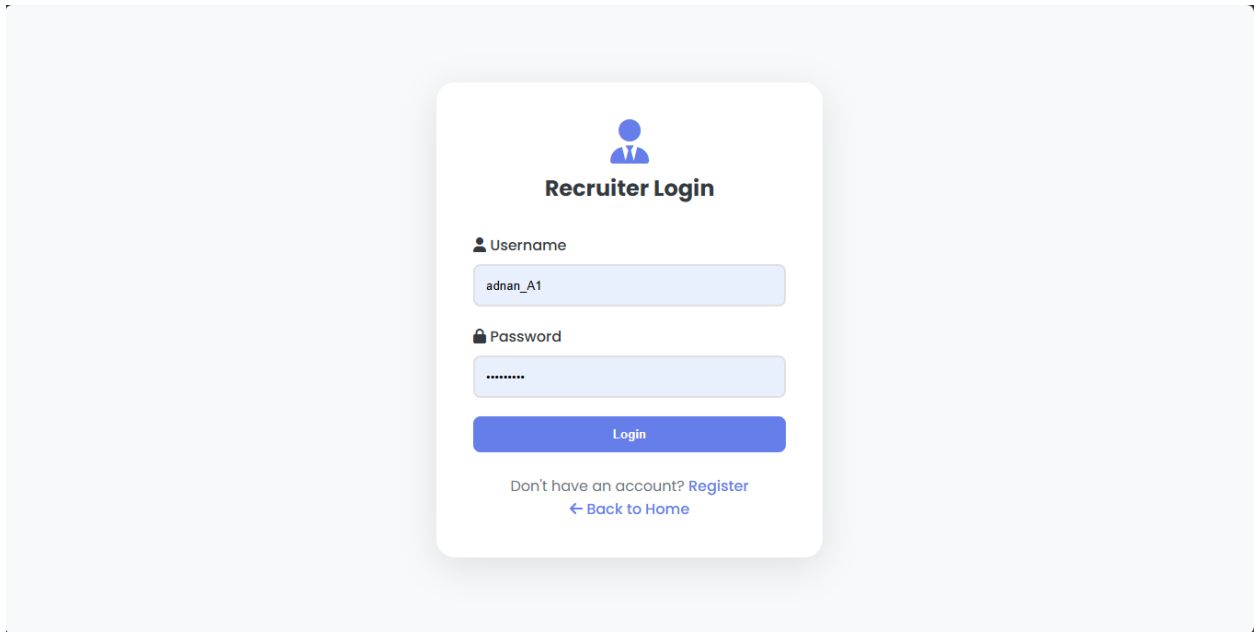


Fig. 3 Landing Page



The image shows a mobile app interface for a Recruiter Login page. It features a white card with rounded corners on a light gray background. At the top of the card is a blue icon of a person in a suit, followed by the title "Recruiter Login" in bold black text. Below the title are two input fields: "Username" with the text "adnan_A1" and "Password" with masked characters "*****". A blue "Login" button is positioned below the password field. At the bottom of the card, there is a link "Don't have an account? Register" and a link "← Back to Home".

Recruiter Login

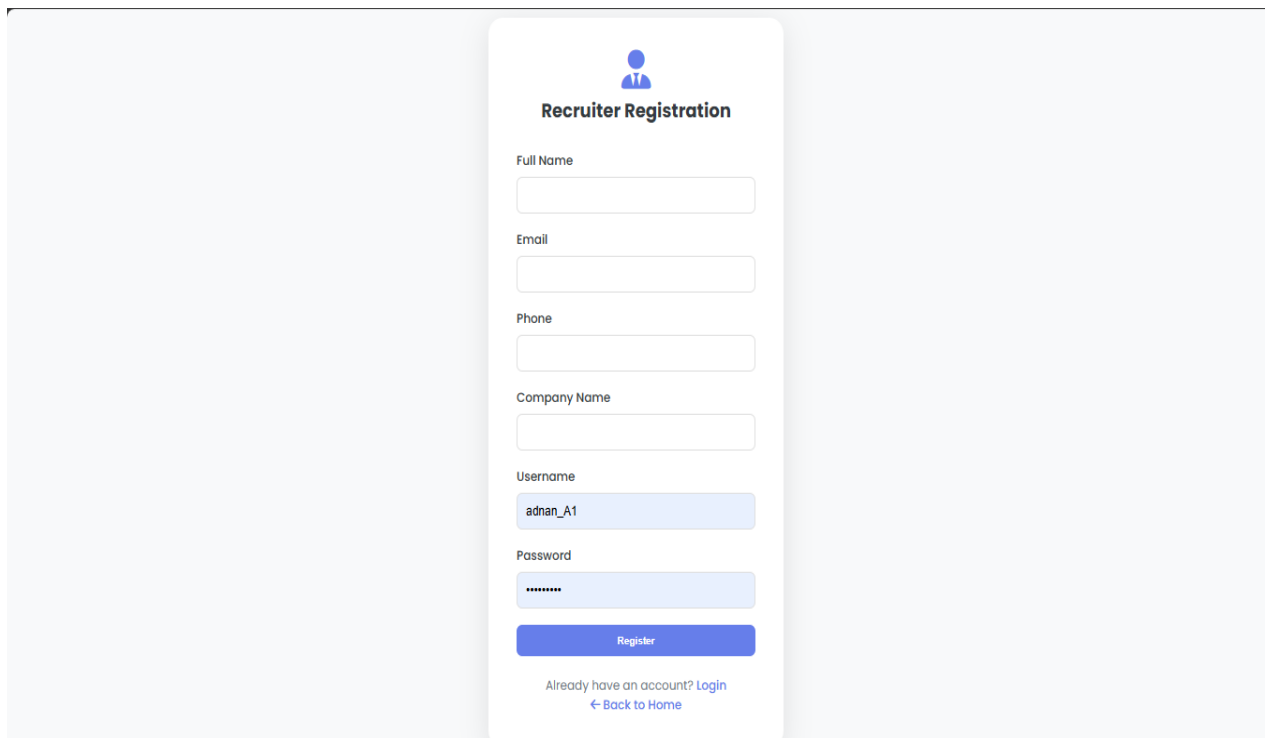
Username
adnan_A1

Password

Login

Don't have an account? [Register](#)
[← Back to Home](#)

Fig. 4 Recruiter Login page



The image shows a mobile app interface for a Recruiter Registration page. It features a white card with rounded corners on a light gray background. At the top of the card is a blue icon of a person in a suit, followed by the title "Recruiter Registration" in bold black text. Below the title are five input fields: "Full Name", "Email", "Phone", "Company Name", and "Username" (with the text "adnan_A1"). Below the "Username" field is a "Password" field with masked characters "*****". A blue "Register" button is positioned below the password field. At the bottom of the card, there is a link "Already have an account? Login" and a link "← Back to Home".

Recruiter Registration

Full Name

Email

Phone

Company Name

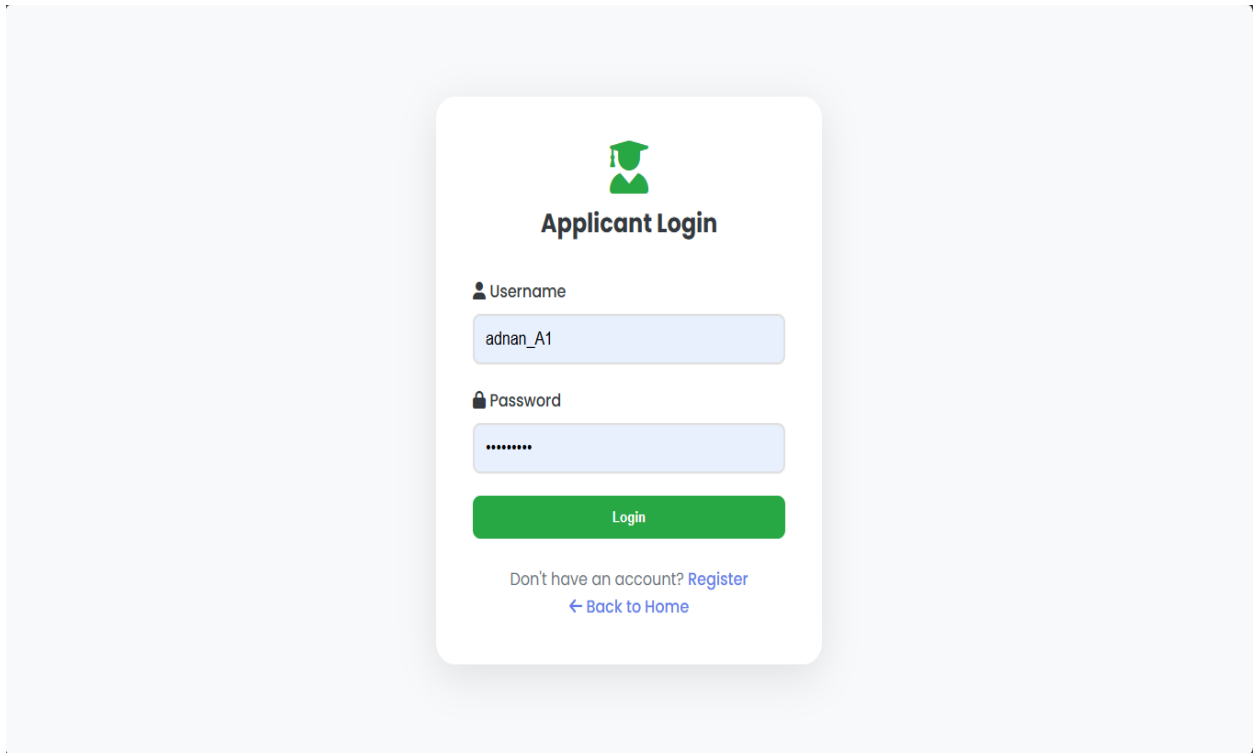
Username
adnan_A1

Password

Register

Already have an account? [Login](#)
[← Back to Home](#)

Fig.5 Recruiter Registration Page



The image shows a login form titled "Applicant Login" with a green graduation cap icon. It includes fields for Username (filled with "adnan_A1") and Password (masked with "*****"). A green "Login" button is at the bottom. Below the button are links for "Register" and "Back to Home".


Applicant Login

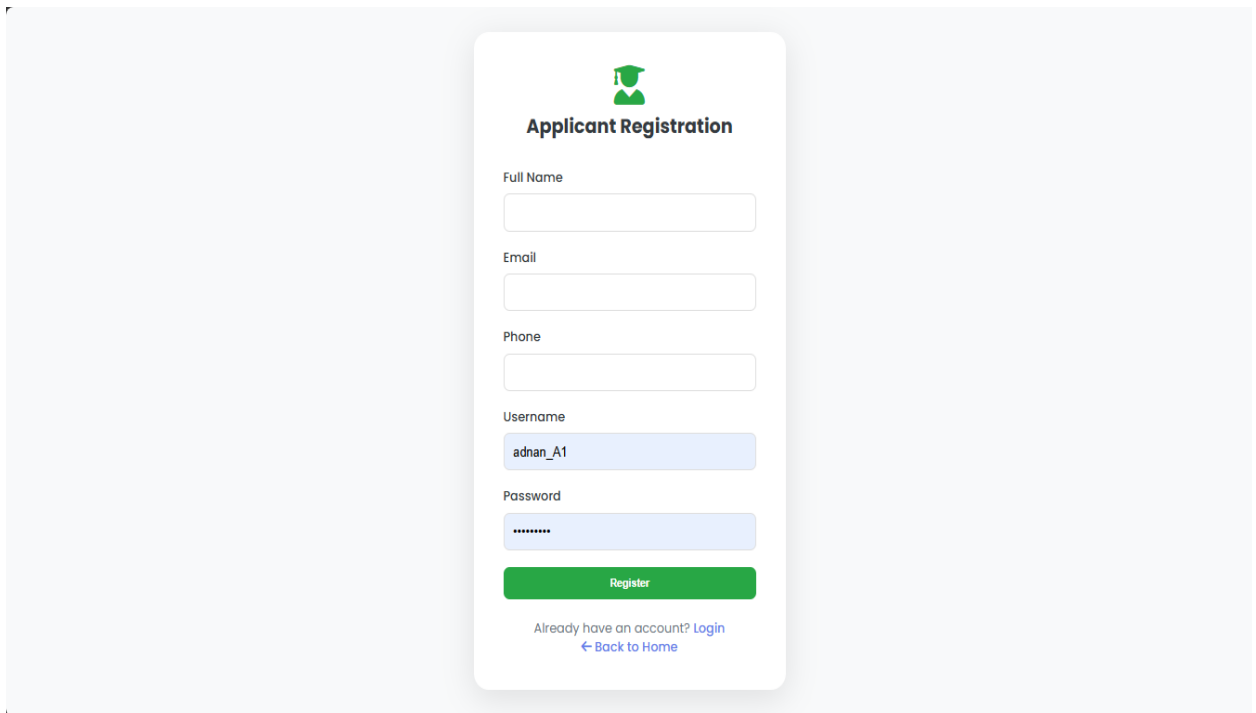
Username

Password

Login

Don't have an account? [Register](#)
[← Back to Home](#)

Fig.6 Applicant Login Page



The image shows a registration form titled "Applicant Registration" with a green graduation cap icon. It includes fields for Full Name, Email, Phone, Username (filled with "adnan_A1"), and Password (masked with "*****"). A green "Register" button is at the bottom. Below the button are links for "Login" and "Back to Home".


Applicant Registration

Full Name

Email

Phone

Username

Password

Register

Already have an account? [Login](#)
[← Back to Home](#)

Fig.6 Applicant Registration Page

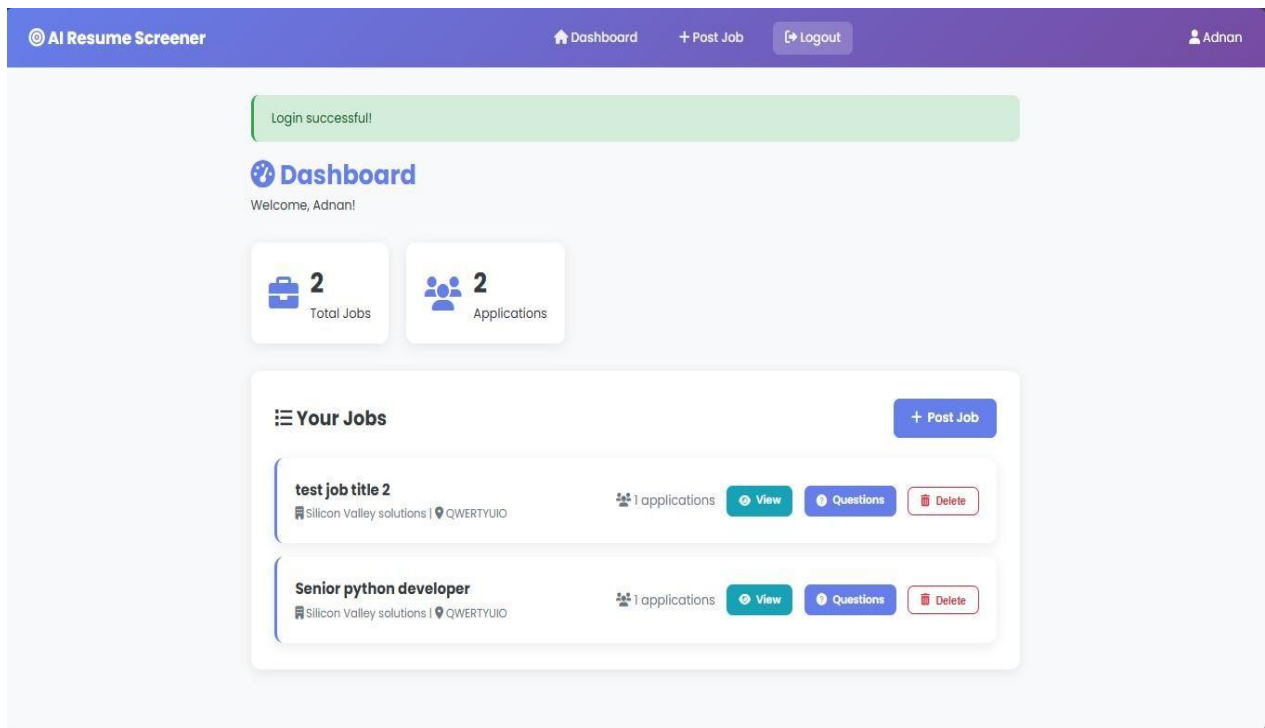


Fig.7 Recruiter Dashboard Page

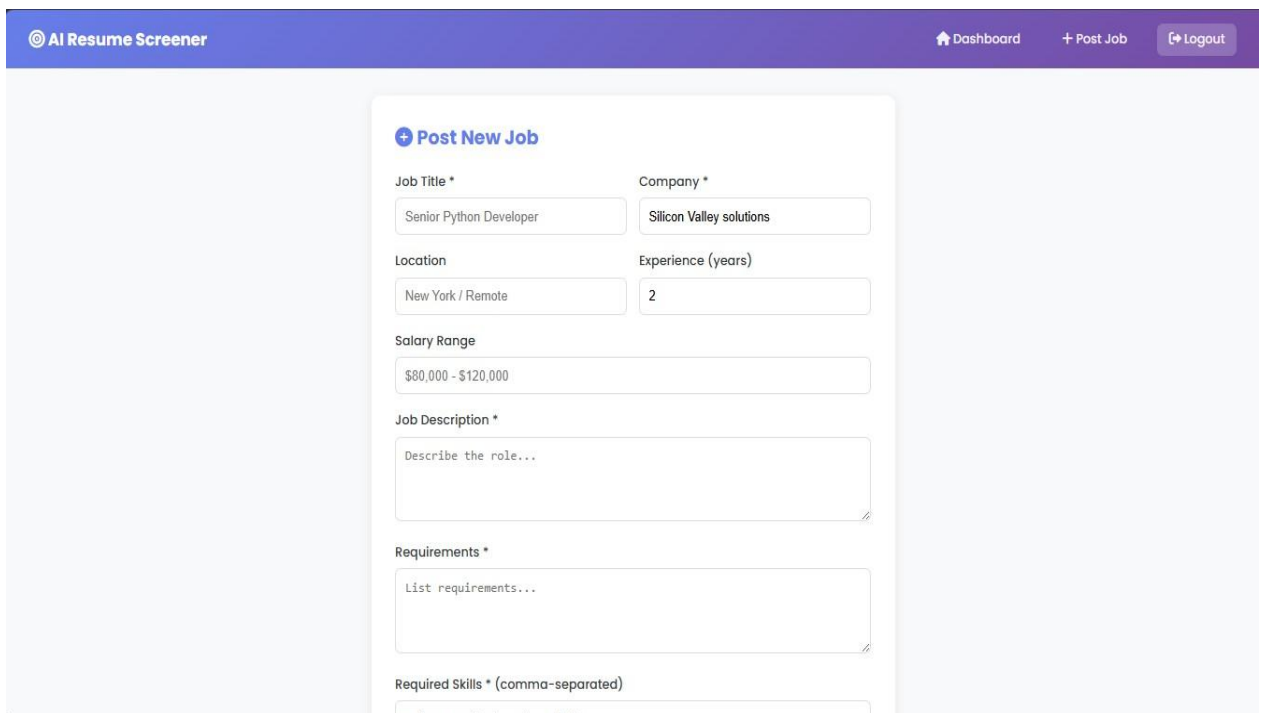


Fig. 8 Job Posting Page

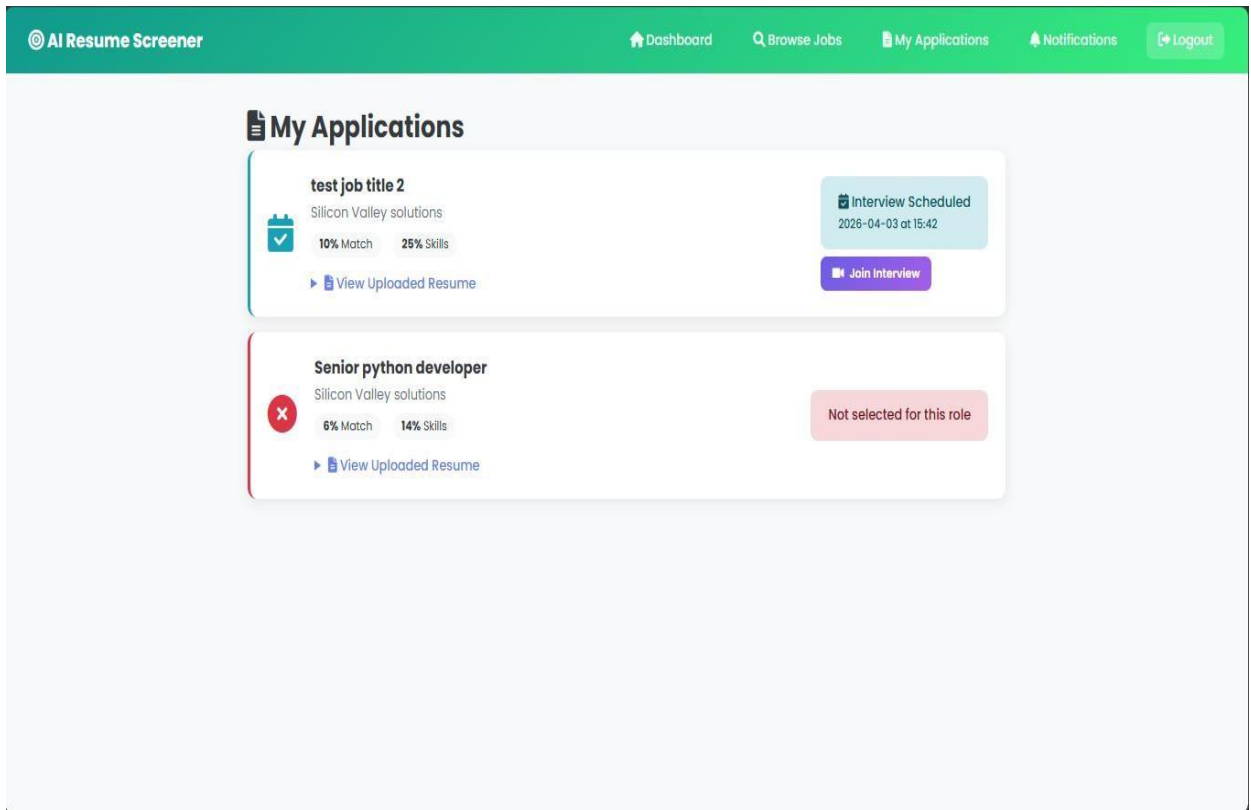


Fig. 9 My Applications Page

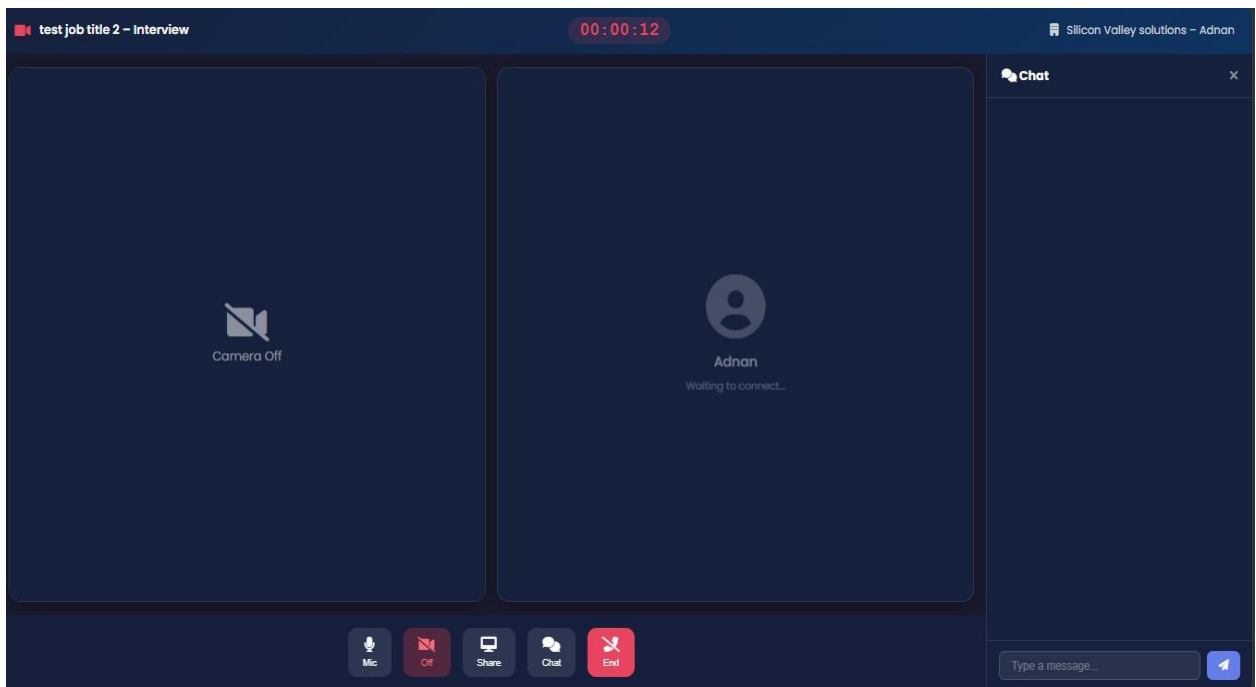


Fig. 10 Interview Room Interface

CHAPTER 4

RESULT ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The implementation of the AI Resume Screening System was carried out through the integration of multiple interconnected modules, including backend development, frontend design, database management, and AI-based processing. The development process followed a structured approach to ensure that each component functions efficiently both independently and as part of the overall system. The backend was developed using the Flask framework, which handled all server-side operations such as routing, request handling, business logic execution, and communication between different system components. Flask provided a flexible and lightweight environment for building the application while ensuring scalability and maintainability.

The frontend of the system was designed using HTML and CSS, focusing on simplicity, responsiveness, and ease of use. The interface was carefully structured to provide a smooth and intuitive experience for both applicants and recruiters. Applicants can easily navigate through job listings, upload resumes, and track application status, while recruiters can manage job postings, review candidates, and schedule interviews through a clean and organized dashboard.

The database was implemented using SQLite, which provides a simple yet effective solution for storing structured data. It maintains records of users, job postings, applications, and interview-related information. The database design ensures efficient data retrieval and storage, supporting the smooth functioning of the system. Proper relationships between tables were established to maintain data integrity and consistency throughout the application.

The complete workflow of the system begins with user authentication, where both applicants and recruiters register and log in to access their respective functionalities. Recruiters can post job openings by providing job descriptions and required skills, while applicants can browse available jobs and apply by uploading their resumes. Once a resume is uploaded, the system processes the file by extracting textual content, anonymizing sensitive information, and applying AI-based algorithms to evaluate the candidate. The TF-IDF and cosine similarity techniques are used to calculate a match score, which reflects the relevance of the candidate's profile to the job requirements. This score, along with other application details, is stored in the database and presented to recruiters in a ranked format, enabling quick and efficient decision-making. This integrated implementation ensures seamless communication between system components and

provides a smooth and efficient recruitment workflow.

4.1.1 Backend Testing – Flask API Routes

Backend testing was conducted extensively to ensure the correctness, reliability, and stability of all Flask routes implemented in the system. Each route was tested individually to verify that it performs its intended function accurately and handles different types of inputs effectively. Authentication routes were thoroughly tested to ensure secure user registration and login processes. Special attention was given to password hashing using bcrypt and session management to maintain secure user authentication.

Job management routes were tested to confirm that recruiters can successfully create, update, and delete job postings without errors. Application-related routes were validated to ensure that resume uploads are handled correctly, text extraction is performed accurately, and application data is stored properly in the database. The AI processing route was a critical component and was tested with various resumes and job descriptions to evaluate the accuracy and consistency of the matching algorithm.

In addition to functionality testing, error handling mechanisms were also evaluated. The system was tested for scenarios such as invalid inputs, missing files, incorrect login credentials, and database errors. Appropriate error messages and responses were verified to ensure that users receive clear and helpful feedback. Overall, backend testing confirmed that all API routes are functioning correctly and reliably under different conditions.

4.1.2 Integration with Frontend

The integration of frontend and backend components was a crucial step in ensuring the proper functioning of the system. Flask's `render_template` function was used to dynamically connect backend data with HTML templates, allowing real-time data to be displayed on the user interface. User inputs from forms were captured and sent to the backend through HTTP POST requests, where they were processed by the corresponding Flask routes.

The system ensures smooth and continuous data flow between the frontend and backend. For instance, when an applicant uploads a resume, the file is transmitted to the server, processed using AI algorithms, and the resulting match score is displayed on the same interface without requiring complex interactions. Similarly, recruiters can view ranked candidate lists, manage applications, schedule interviews, and provide feedback through interactive dashboards.

This integration was thoroughly tested to ensure that all user actions are accurately reflected in the system. Special care was taken to eliminate delays, inconsistencies, or synchronization issues between the frontend and backend components. The successful integration ensures a seamless and responsive user experience.

4.1.3 Resume Matching Accuracy

The accuracy of the resume matching system was evaluated based on its ability to correctly identify and rank candidates according to their suitability for a given job role. The system uses TF-IDF vectorization to convert textual data into numerical form and cosine similarity to measure the relevance between resumes and job descriptions. In addition, skill-based matching is performed by comparing extracted skills with those specified in the job requirements.

The combination of these techniques ensures a comprehensive evaluation of candidates by considering both contextual relevance and specific skill sets. A weighted scoring formula is used to calculate the final match score, where textual similarity contributes approximately 60% and skill matching contributes 40%. This balanced approach improves the reliability of the evaluation process.

Testing with multiple resumes demonstrated that candidates possessing relevant skills and experience consistently received higher scores, while less relevant profiles were ranked lower. This indicates that the system effectively distinguishes between suitable and unsuitable candidates, thereby improving the quality of shortlisting and decision-making.

4.2 Testing & Validation

Testing and validation were conducted to evaluate the performance, usability, and reliability of the system under real-world conditions. Different testing strategies were applied to ensure that the system meets both functional and non-functional requirements.

4.2.1 User Testing

User testing was performed by allowing sample users, including both applicants and recruiters, to interact with the system. They were asked to perform common tasks such as registration, job posting, resume uploading, and interview scheduling. Feedback was collected regarding usability, navigation, and overall experience. The results indicated that the system is easy to use and provides a smooth and intuitive interface. Minor improvements, such as enhanced error messages and better layout design, were identified and implemented to further improve user experience.

4.2.2 Load Testing

Load testing was conducted to evaluate the system's performance under multiple user requests. The system was tested with concurrent operations such as multiple resume uploads, job applications, and data retrieval requests. The results showed that the system can handle moderate workloads efficiently without significant delays. Although SQLite is not designed for large-scale enterprise applications, it performs well for small to medium-scale systems. The system can be scaled further by integrating more robust database solutions if required.

4.2.3 Error Rate and Response Time

The system's error rate and response time were analyzed to measure its performance efficiency. The implementation of proper validation and exception handling mechanisms resulted in a low error rate. Most operations, including resume processing, AI matching, and data retrieval, were completed within acceptable time limits. Efficient algorithms and optimized database queries contributed to faster response times, ensuring that users receive quick and reliable feedback.

4.3 Result Interpretation

The results obtained from testing and evaluation demonstrate that the system effectively achieves its intended objectives. The AI-based matching algorithm successfully identifies relevant candidates and ranks them according to their suitability for specific job roles. This significantly reduces the time and effort required for manual resume screening.

The anonymization feature ensures fairness by removing personal identifiers from resumes, thereby minimizing bias in the recruitment process. Additionally, the integration of interview management features enhances the system's functionality, making it a complete and comprehensive recruitment solution. The overall performance indicates improved efficiency, accuracy, and transparency compared to traditional recruitment methods.

4.4 Continuous Feedback and Improvement

Continuous feedback played a vital role in refining and improving the system throughout its development and testing phases. Feedback from users and testing results were carefully analyzed to identify areas for enhancement. Regular updates were made to fix bugs, optimize system performance, and improve the user interface.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The AI Resume Screening System developed in this project successfully addresses the major challenges associated with traditional recruitment processes, which are often time-consuming, inefficient, inconsistent, and prone to bias. In conventional hiring systems, recruiters are required to manually review a large number of resumes, leading to increased workload, delayed decision-making, and the possibility of overlooking highly qualified candidates. The proposed system overcomes these limitations by introducing an automated, intelligent, and structured approach to recruitment. By leveraging modern web technologies and Artificial Intelligence techniques, the system streamlines the entire hiring workflow—from resume submission and analysis to candidate ranking and interview management—thereby significantly improving both efficiency and accuracy.

One of the most significant achievements of this project is the successful integration of Artificial Intelligence techniques for resume analysis and candidate evaluation. The system utilizes TF-IDF (Term Frequency–Inverse Document Frequency) to transform unstructured textual data into meaningful numerical representations, capturing the importance and relevance of words within resumes and job descriptions. This is further enhanced by the application of cosine similarity, which measures the degree of similarity between the candidate’s resume and the job requirements. Unlike traditional keyword-based filtering systems, this approach enables context-aware analysis, ensuring that candidates are evaluated based on the overall relevance of their profiles rather than just the presence of specific keywords. In addition, the inclusion of skill-based matching provides an extra layer of evaluation by comparing candidate skills directly with job requirements. This hybrid evaluation approach results in more accurate, reliable, and meaningful candidate rankings, enabling recruiters to make informed and data-driven hiring decisions.

Another major accomplishment of this system is the implementation of resume anonymization, which plays a crucial role in promoting fairness and ethical hiring practices. By removing sensitive personal information such as names, email addresses, phone numbers, and gender-related identifiers, the system ensures that candidates are evaluated solely on their qualifications, skills.

The project also demonstrates strong database design and data management capabilities. The use of SQLite as the database management system provides a lightweight yet efficient solution for storing and managing recruitment-related data. The database is carefully structured with tables for users, job postings, applications, and interviews, ensuring organized storage and easy retrieval of information. Relationships between these entities are maintained to preserve data consistency and integrity. This structured data management approach enables smooth system operations and supports the seamless execution of various functionalities, including application tracking, candidate ranking, and interview scheduling.

Another important strength of the system lies in its modular and scalable architecture. The application is divided into independent modules such as user authentication, job management, resume processing, AI-based matching, ranking, and interview management. Each module is developed and tested individually, ensuring robustness and reducing the likelihood of system errors. At the same time, these modules are effectively integrated to provide a unified and cohesive system. This modular design not only improves maintainability but also allows for easy upgrades and extensions in the future. New features can be added without disrupting existing functionalities, making the system highly adaptable to evolving requirements.

The user interface of the system has been designed with a strong emphasis on usability, accessibility, and user experience. Applicants are provided with a simple and intuitive platform where they can register, browse job listings, upload resumes, and track their application status with ease. Recruiters, on the other hand, have access to a comprehensive dashboard that allows them to post job openings, review applications, analyze candidate rankings, and manage interviews efficiently. The system ensures smooth navigation and minimizes complexity, making it suitable for users with varying levels of technical expertise. Furthermore, the integration of a notification system enhances communication by providing real-time updates on application status, interview schedules, and selection outcomes.

Extensive testing and validation have been carried out to ensure the reliability, performance, and accuracy of the system. The results indicate that the AI-based matching algorithm consistently produces accurate and relevant candidate rankings. The system demonstrates a low error rate due to effective input validation and error-handling mechanisms. Response times for key operations

such as resume processing, similarity computation, and data retrieval are within acceptable limits, ensuring a smooth and responsive user experience. Load testing confirms that the system can handle multiple concurrent users and operations without significant performance degradation, making it suitable for small to medium-scale deployment scenarios.

In addition to its technical strengths, the system contributes significantly to improving transparency and accountability in the recruitment process. By providing clear match scores, structured evaluation criteria, and ranked candidate lists, it allows recruiters to understand the basis of candidate selection. At the same time, applicants gain insights into how their profiles are evaluated, which helps them identify areas for improvement. This transparency fosters trust between users and enhances the overall effectiveness of the hiring process.

Moreover, the system demonstrates the practical application of Artificial Intelligence and web development technologies in solving real-world problems. It highlights how intelligent automation can be used to optimize complex processes such as recruitment, which involve large volumes of unstructured data and require careful decision-making. The project serves as a strong example of how theoretical concepts such as Natural Language Processing, similarity analysis, and database management can be effectively implemented in a real-world application.

The system also lays a solid foundation for future enhancements and scalability. Its flexible architecture allows for integration with advanced AI models, external job platforms, and cloud-based services. Features such as real-time analytics, predictive hiring insights, and automated candidate recommendations can be incorporated in future versions. This adaptability ensures that the system remains relevant in the rapidly evolving field of digital recruitment.

In conclusion, the AI Resume Screening System represents a comprehensive, efficient, and intelligent solution to modern recruitment challenges. It successfully combines automation, Artificial Intelligence, and user-centric design to create a platform that improves hiring efficiency, enhances accuracy, and promotes fairness. By reducing manual effort, minimizing bias, and enabling data-driven decision-making, the system significantly transforms the recruitment process. Furthermore, it demonstrates the potential of integrating AI technologies into everyday applications, paving the way for more advanced and innovative solutions in the future.

5.1 Future Enhancements

Although the current AI Resume Screening System is functional, efficient, and capable of addressing the core challenges of modern recruitment, there remains significant scope for future enhancements. These improvements can further strengthen the system's intelligence, scalability, usability, and overall impact, transforming it into a more advanced, adaptive, and enterprise-ready recruitment platform. As the demand for faster, smarter, and more data-driven hiring solutions continues to grow, incorporating emerging technologies and expanding system capabilities will be essential to ensure long-term relevance and competitiveness. By continuously evolving, the system can effectively meet the complex requirements of large-scale, dynamic, and globally distributed hiring environments.

One of the most important areas for future enhancement is the integration of advanced Artificial Intelligence models. While the current system uses TF-IDF and cosine similarity for efficient and lightweight matching, these techniques can be further enhanced by incorporating deep learning and transformer-based Natural Language Processing models such as BERT, RoBERTa, or GPT-based architectures. These models are capable of understanding the contextual, semantic, and syntactic meaning of text at a much deeper level. Unlike traditional approaches, they can capture relationships between words, interpret intent, and identify subtle patterns within resumes and job descriptions. As a result, the system would be able to perform highly accurate candidate-job matching, detect transferable skills, and provide more personalized and relevant recommendations. This advancement would significantly improve the quality of hiring decisions and move the system toward true intelligent recruitment automation.

Another significant improvement involves integration with external professional and learning platforms. By connecting the system with APIs from platforms such as LinkedIn, it would be possible to automatically fetch candidate profiles, work experience, endorsements, and professional networks. This would reduce the need for manual data entry and ensure that candidate information is always up-to-date and comprehensive. Additionally, integration with online learning platforms such as Google Learning APIs, Coursera, or Udemy can enhance the system's functionality by recommending relevant courses and certifications. For instance, if the system identifies missing or weak skills in a candidate's profile, it can suggest targeted learning resources to help bridge those gaps. This not only improves candidate employability but also transforms the system into a career development and guidance platform rather than just a recruitment tool.

The addition of a personalized AI-powered chatbot assistant is another promising enhancement that can significantly improve user interaction and engagement. A chatbot integrated within the system can provide real-time assistance to both applicants and recruiters. For applicants, it can guide them through the process of creating resumes, suggest improvements based on job requirements, answer frequently asked questions, and provide updates on application status. For recruiters, the chatbot can assist in posting jobs, filtering candidates, generating interview questions, and scheduling interviews. Advanced chatbots can also use conversational AI to provide intelligent recommendations and automate repetitive tasks, thereby improving efficiency and reducing human effort. This feature would make the system more interactive, user-friendly, and accessible to a wider audience.

Scalability is another crucial aspect that can be enhanced in future versions of the system. While SQLite serves as a simple and effective database for small to medium-scale applications, large-scale deployment would require more robust database management systems such as MySQL or PostgreSQL. These databases offer better performance, support for concurrent users, and advanced data management capabilities. In addition, migrating the system to cloud-based platforms such as AWS, Google Cloud, or Microsoft Azure can significantly improve reliability, availability, and scalability. Cloud deployment enables features such as load balancing, automatic scaling, distributed storage, and data backup, ensuring that the system can handle high traffic and large volumes of data without performance degradation.

Future Enhancements for AI Resume Screening System

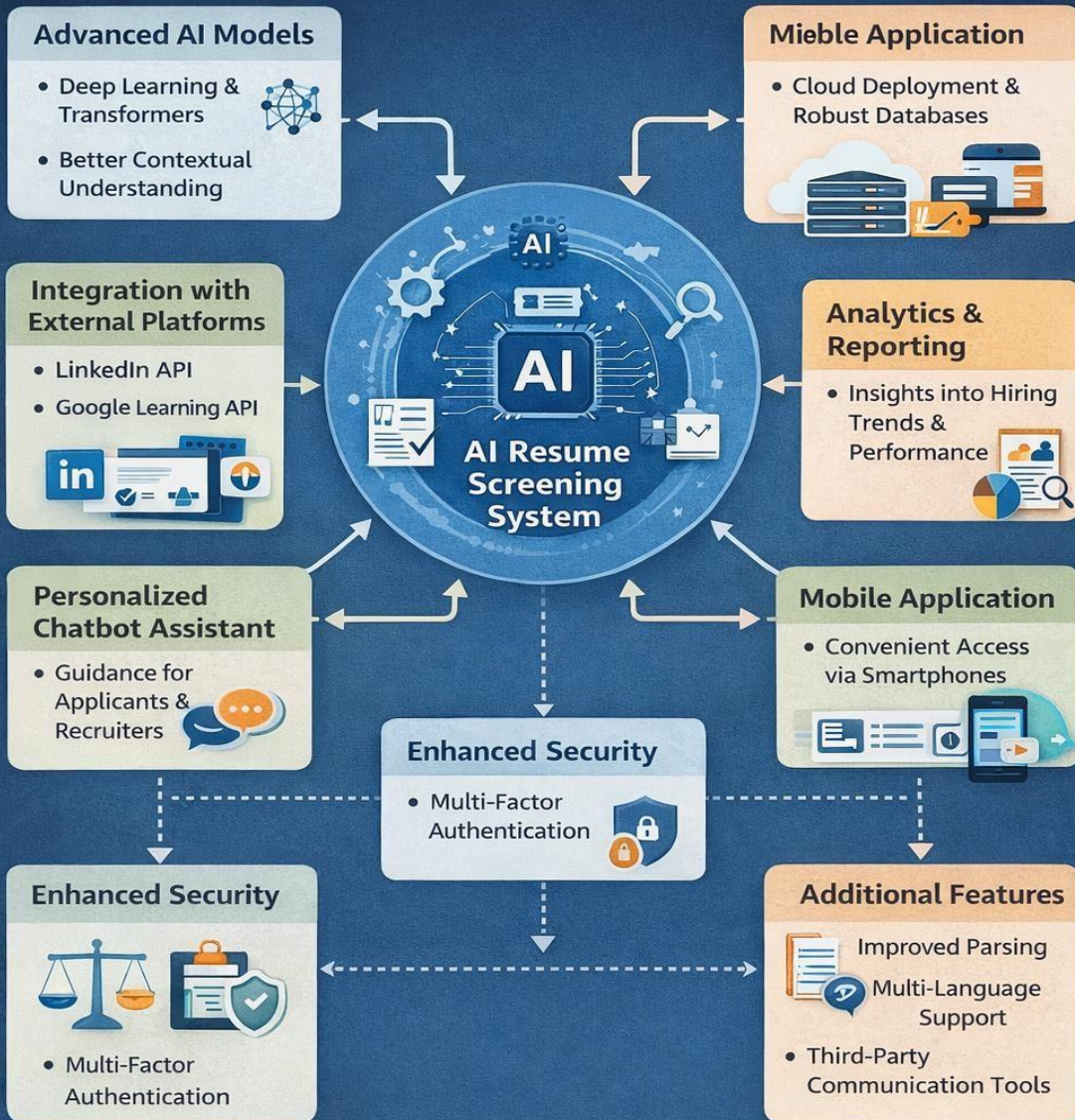


Fig.11 Future Enhancements

APPENDIX A:

A.1 Introduction to Source Code

This appendix presents the core implementation of the AI Resume Screening System. The system is developed using Python and the Flask framework, integrating multiple modules such as user authentication, resume processing, AI-based matching, and database operations.

The source code demonstrates how different components of the system interact to perform tasks such as:

- User registration and login
- Resume upload and text extraction
- AI-based resume-job matching
- Application storage and ranking

The implementation follows a modular approach, ensuring clarity, maintainability, and scalability.

A.2 Key Functional Components

The main functionalities implemented in the code include:

- **Authentication Module:** Handles user login and registration securely using password hashing
- **Resume Processing Module:** Extracts and preprocesses text from PDF resumes
- **AI Matching Module:** Uses TF-IDF and cosine similarity for intelligent matching
- **Database Module:** Stores user, job, and application data
- **API Routes:** Enable communication between frontend and backend

////

////

```
os.makedirs('uploads', exist_ok=True)
```

```
# Skills database
```

```
SKILLS = [  
    'python', 'java', 'javascript', 'c++', 'c#', 'html', 'css', 'react', 'angular',  
    'node.js', 'django', 'flask', 'sql', 'mysql', 'mongodb', 'aws', 'docker',  
    'kubernetes', 'machine learning', 'deep learning', 'nlp', 'tensorflow',  
    'pytorch', 'pandas', 'numpy', 'git', 'linux', 'api', 'rest', 'agile'  
]
```

```
GENDER_WORDS = ['he', 'she', 'him', 'her', 'his', 'hers', 'male', 'female', 'mr', 'mrs', 'ms']
```

```
# ===== DATABASE =====
```

```
def get_db():
```

```
    conn = sqlite3.connect(DATABASE)
```

```
    conn.row_factory = sqlite3.Row
```

```
    return conn
```

```
def init_db():
```

```
    conn = get_db()
```

```
    conn.executescript("""
```

```
        CREATE TABLE IF NOT EXISTS users (
```

```
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
            username TEXT UNIQUE,
```

```

    password TEXT,
    email TEXT,
    fullname TEXT,
    phone TEXT,
    role TEXT,
    company TEXT
);

CREATE TABLE IF NOT EXISTS jobs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    recruiter_id INTEGER,
    title TEXT,
    company TEXT,
    location TEXT,
    description TEXT,
    requirements TEXT,
    skills TEXT,
    experience INTEGER,
    salary TEXT,
    status TEXT DEFAULT 'active'
);

CREATE TABLE IF NOT EXISTS applications (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    job_id INTEGER,
    applicant_id INTEGER,

```

```

    resume_text TEXT,
    anon_text TEXT,
    match_score REAL,
    skill_score REAL,
    matched_skills TEXT,
    missing_skills TEXT,
    experience INTEGER,
    status TEXT DEFAULT 'pending',
    UNIQUE(job_id, applicant_id)
);

CREATE TABLE IF NOT EXISTS notifications (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    title TEXT,
    message TEXT,
    type TEXT,
    is_read INTEGER DEFAULT 0
);

")
conn.commit()
conn.close()

init_db()

```

```

# ===== HELPERS =====

def allowed_file(filename):

    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS


def hash_password(password):

    return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()


def check_password(password, hashed):

    return bcrypt.checkpw(password.encode(), hashed.encode())


def extract_text_from_file(filepath):

    """Extract text from uploaded resume"""

    ext = filepath.split('.')[-1].lower()

    try:

        if ext == 'txt':

            with open(filepath, 'r', encoding='utf-8', errors='ignore') as f:

                return f.read()

        elif ext == 'pdf':

            import PyPDF2

            with open(filepath, 'rb') as f:

                reader = PyPDF2.PdfReader(f)

                text = ""

                for page in reader.pages:

                    text += page.extract_text() or ""

```

```

        return text

    elif ext == 'docx':

        from docx import Document

        doc = Document(filepath)

        return ''.join([p.text for p in doc.paragraphs])

    except:

        pass

    return ""

def anonymize_text(text):

    """Remove name, gender, contact info using REGEX"""

    # Remove emails

    text = re.sub(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}', '[EMAIL]', text)

    # Remove phones

    text = re.sub(r'\+?\d{1,3}[-.\s]?(\d{3})?[-.\s]?d{3}[-.\s]?d{4}', '[PHONE]', text)

    text = re.sub(r'\bd{10,12}\b', '[PHONE]', text)

    # Remove gender words

    for word in GENDER_WORDS:

        text = re.sub(r'\b' + word + r'\b', '', text, flags=re.IGNORECASE)

    # Remove names (first 3 lines, capitalized words)

    lines = text.split('\n')

    skip = ['Resume', 'CV', 'Summary', 'Profile', 'Experience', 'Education', 'Skills']

    for i in range(min(3, len(lines))):

        for word in re.findall(r'\b[A-Z][a-z]{2,}\b', lines[i]):

```



```

        if word not in skip:

            lines[i] = lines[i].replace(word, '[NAME]')

    return '\n'.join(lines)

def calculate_match(resume_text, job_desc, required_skills):

    """Calculate match score using TF-IDF and skill matching"""

    # TF-IDF Similarity

    try:

        vectorizer = TfidfVectorizer(stop_words='english')

        matrix = vectorizer.fit_transform([resume_text.lower(), job_desc.lower()])

        similarity = cosine_similarity(matrix[0:1], matrix[1:2])[0][0]

    except:

        similarity = 0

    # Skill matching

    resume_lower = resume_text.lower()

    found_skills = [s for s in SKILLS if re.search(r'\b' + re.escape(s) + r'\b', resume_lower)]

    required_set = set(s.lower() for s in required_skills)

    matched = list(required_set & set(found_skills))

    missing = list(required_set - set(found_skills))

    skill_pct = (len(matched) / len(required_set) * 100) if required_set else 0

    # Experience

    exp_match = re.search(r'(\d+)\s*(?:years?|yrs?)', resume_lower)

```

```
experience = int(exp_match.group(1)) if exp_match else 0
```

```
# Overall score
```

```
overall = (similarity * 60) + (skill_pct * 0.4)
```

```
return {
```

```
    'match_score': round(overall, 1),
```

```
    'skill_score': round(skill_pct, 1),
```

```
    'matched': matched,
```

```
    'missing': missing,
```

```
    'experience': experience,
```

```
    'all_skills': found_skills
```

```
}
```

```
# ===== ROUTES =====
```

```
@app.route('/')
```

```
def landing():
```

```
    if 'user_id' in session:
```

```
        if session['role'] == 'recruiter':
```

```
            return redirect(url_for('recruiter_dashboard'))
```

```
            return redirect(url_for('applicant_dashboard'))
```

```
    return render_template('landing.html')
```

```

# ----- RECRUITER AUTH -----

@app.route('/recruiter/login', methods=['GET', 'POST'])
def recruiter_login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        conn = get_db()
        user = conn.execute('SELECT * FROM users WHERE username=? AND role=?',
                             (username, 'recruiter')).fetchone()
        conn.close()

        if user and check_password(password, user['password']):
            session['user_id'] = user['id']
            session['username'] = user['username']
            session['fullname'] = user['fullname']
            session['role'] = 'recruiter'
            session['company'] = user['company']
            flash('Login successful!', 'success')
            return redirect(url_for('recruiter_dashboard'))

        flash('Invalid credentials', 'danger')
        return render_template('recruiter_login.html')

@app.route('/recruiter/register', methods=['GET', 'POST'])

```

```

def recruiter_register():
    if request.method == 'POST':
        try:
            conn = get_db()

            conn.execute("""INSERT INTO users (username, password, email, fullname, phone, role,
company)

                VALUES (?, ?, ?, ?, ?, ?, ?)""",

            (request.form['username'], hash_password(request.form['password']),
            request.form['email'], request.form['fullname'],
            request.form['phone'], 'recruiter', request.form['company']))

            conn.commit()

            conn.close()

            flash('Registration successful! Please login.', 'success')

            return redirect(url_for('recruiter_login'))

        except sqlite3.IntegrityError:

            flash('Username already exists', 'danger')

            return render_template('recruiter_register.html')

# ----- APPLICANT AUTH -----

@app.route('/applicant/login', methods=['GET', 'POST'])
def applicant_login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

```

```

conn = get_db()

user = conn.execute('SELECT * FROM users WHERE username=? AND role=?',
                    (username, 'applicant')).fetchone()

conn.close()

if user and check_password(password, user['password']):
    session['user_id'] = user['id']
    session['username'] = user['username']
    session['fullname'] = user['fullname']
    session['role'] = 'applicant'

    flash('Login successful!', 'success')

    return redirect(url_for('applicant_dashboard'))

flash('Invalid credentials', 'danger')

return render_template('applicant_login.html')

@app.route('/applicant/register', methods=['GET', 'POST'])
def applicant_register():
    if request.method == 'POST':
        try:
            conn = get_db()

            conn.execute("""INSERT INTO users (username, password, email, fullname, phone, role)
                           VALUES (?, ?, ?, ?, ?, ?)""",
                        (request.form['username'], hash_password(request.form['password']),

```

```

        request.form['email'], request.form['fullname'],
        request.form['phone'], 'applicant'))

    conn.commit()

    conn.close()

    flash('Registration successful! Please login.', 'success')

    return redirect(url_for('applicant_login'))

except sqlite3.IntegrityError:

    flash('Username already exists', 'danger')

    return render_template('applicant_register.html')


@app.route('/logout')
def logout():

    session.clear()

    flash('Logged out', 'success')

    return redirect(url_for('landing'))


# ----- RECRUITER PAGES -----

@app.route('/recruiter/dashboard')
def recruiter_dashboard():

    if session.get('role') != 'recruiter':

        return redirect(url_for('landing'))


    conn = get_db()

    jobs = conn.execute("""SELECT j.*,"

```

```

        (SELECT COUNT(*) FROM applications WHERE job_id=j.id) as app_count
        FROM jobs j WHERE recruiter_id=? ORDER BY id DESC",
        (session['user_id'],)).fetchall()

conn.close()

total_apps = sum(j['app_count'] for j in jobs)

return render_template('recruiter_dashboard.html', jobs=jobs, total_apps=total_apps)

@app.route('/recruiter/post-job', methods=['GET', 'POST'])
def post_job():
    if session.get('role') != 'recruiter':
        return redirect(url_for('landing'))

    if request.method == 'POST':
        skills = [s.strip().lower() for s in request.form['skills'].split(',') if s.strip()]

        conn = get_db()

        conn.execute("""INSERT INTO jobs (recruiter_id, title, company, location, description,
            requirements, skills, experience, salary)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
            (session['user_id'], request.form['title'], request.form['company'],
            request.form['location'], request.form['description'],
            request.form['requirements'], json.dumps(skills),
            int(request.form.get('experience', 0)), request.form.get('salary', "")))

```

```

    conn.commit()

    conn.close()

    flash('Job posted successfully!', 'success')

    return redirect(url_for('recruiter_dashboard'))


return render_template('post_job.html', company=session.get('company', ''))


@app.route('/recruiter/applications/<int:job_id>')
def view_applications(job_id):
    if session.get('role') != 'recruiter':
        return redirect(url_for('landing'))

    conn = get_db()

    job = conn.execute('SELECT * FROM jobs WHERE id=?', (job_id,)).fetchone()

    apps = conn.execute("""SELECT * FROM applications WHERE job_id=?
                        ORDER BY match_score DESC""", (job_id,)).fetchall()

    conn.close()

    if not job:
        flash('Job not found', 'danger')

        return redirect(url_for('recruiter_dashboard'))

    job = dict(job)

    job['skills'] = json.loads(job['skills'])

```



```

    job_list = []

    for j in jobs:

        job = dict(j)

        job['skills'] = json.loads(job['skills'])

        job_list.append(job)

    return render_template('browse_jobs.html', jobs=job_list, applied_ids=applied_ids)

@app.route('/applicant/apply/<int:job_id>', methods=['GET', 'POST'])
def apply_job(job_id):

    if session.get('role') != 'applicant':

        return redirect(url_for('landing'))

    conn = get_db()

    job = conn.execute('SELECT * FROM jobs WHERE id=?', (job_id,)).fetchone()

    conn.close()

    if not job:

        flash('Job not found', 'danger')

        return redirect(url_for('browse_jobs'))

    job = dict(job)

    job['skills'] = json.loads(job['skills'])

```

result = None

applied = False

if request.method == 'POST':

if 'resume' not in request.files:

flash('No file uploaded', 'danger')

else:

file = request.files['resume']

if file and allowed_file(file.filename):

filename = secure_filename(f'{session["user_id"]}_{job_id}_{file.filename}')

filepath = os.path.join(UPLOAD_FOLDER, filename)

file.save(filepath)

REFERENCES

1. Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*. 4th Edition, Pearson Education, 2021.
2. Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press, 2016.
3. Jurafsky, D., & Martin, J. H. *Speech and Language Processing*. 3rd Edition (Draft), Stanford University, 2023.
4. Manning, C. D., Raghavan, P., & Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
5. Pedregosa, F., et al. “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825–2830.
6. Reimers, N., & Gurevych, I. “Sentence-BERT: Sentence Embeddings using Siamese

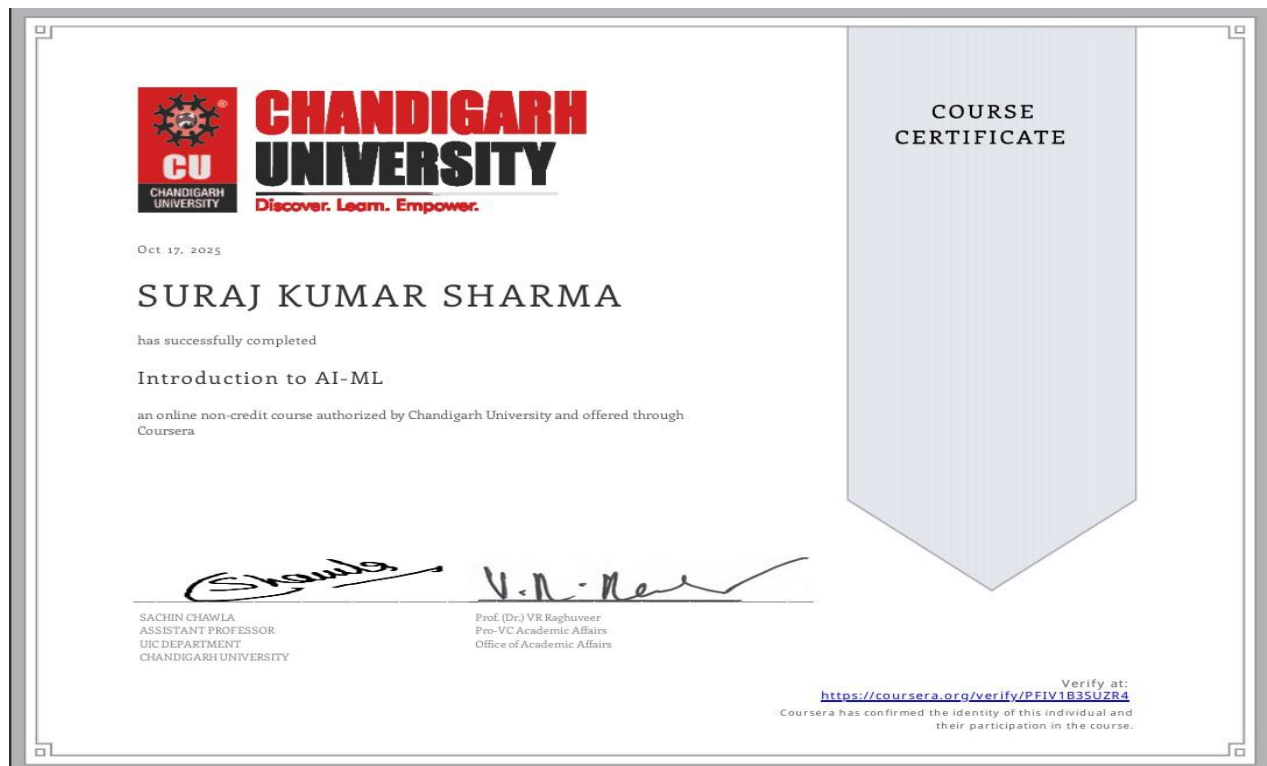
BERT-Networks.” *Proceedings of EMNLP*, 2019.

7. Bird, S., Klein, E., & Loper, E. *Natural Language Processing with Python*. O’Reilly Media, 2009.
8. Flask Documentation. “Flask Web Framework.” Available at:
<https://flask.palletsprojects.com/>
9. Scikit-learn Documentation. “TF-IDF Vectorizer and Cosine Similarity.” Available at:
<https://scikit-learn.org/>
10. PyPDF2 Documentation. “PDF Text Extraction in Python.” Available at:
<https://pypdf2.readthedocs.io/>
11. SQLite Documentation. “SQLite Database Engine.” Available at:
<https://www.sqlite.org/docs.html>
12. Bcrypt Documentation. “Password Hashing in Python.” Available at:
<https://pypi.org/project/bcrypt/>
13. LinkedIn Talent Solutions. “Global Recruiting Trends Report.” LinkedIn, 2022.
14. Glassdoor. “Recruitment Statistics and Hiring Trends.” 2023.
15. IEEE Research Paper: “Automated Resume Screening Using Natural Language Processing,” IEEE Xplore, 2020.

Git Hub Link: <https://github.com/surajsharma-ai/Resume-Screening-System>

Certificates:

a) Suraj Sharma



b) Adnan Irshad

